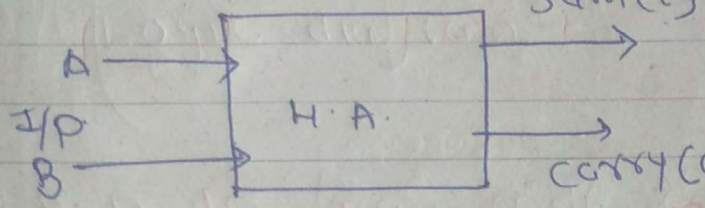
Adders :- Add two binary numbers.

Types of Binary adders :-
1) Half adder
2) Full adder.

Half adder →

Two i/p = A, B
Two o/p = sum, carry.



I/P		O/P	
A	B	Sum	carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

ⓐ Block diagram

k-map

Sum

A \ B	0	1
0	0	(1)
1	(1)	0

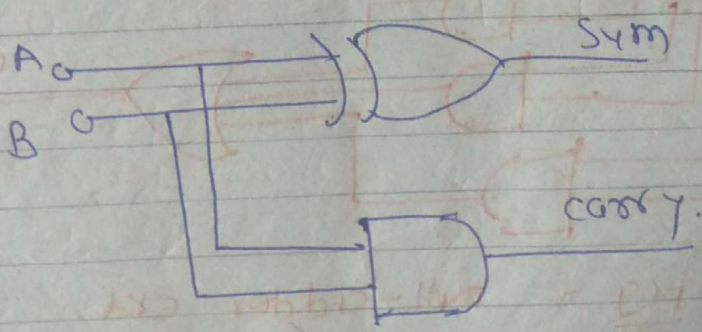
Carry

A \ B	0	1
0	0	0
1	0	(1)

$S = A\bar{B} + \bar{B}A$

$S = A \oplus B$

$carry = A \cdot B$

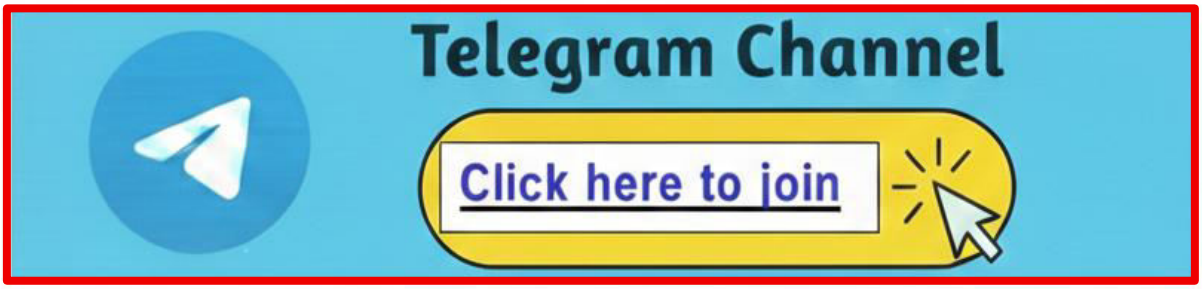


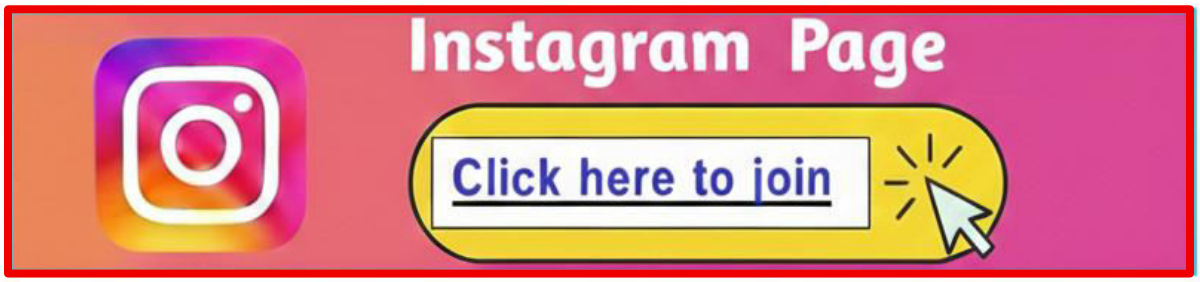
Full adder

https://t.me/SPPU_TE_BE_COMP

Scanned by CamScanner

PDF BY VL



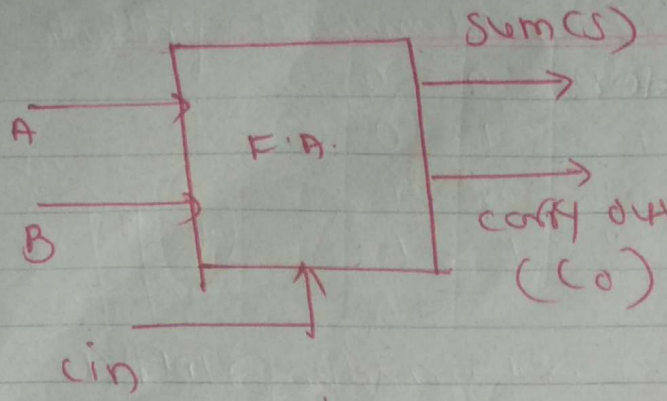






© 2026 Vishwajeet Londhe. All Rights Reserved

https://t.me/SPPU_TE_BE_COMP

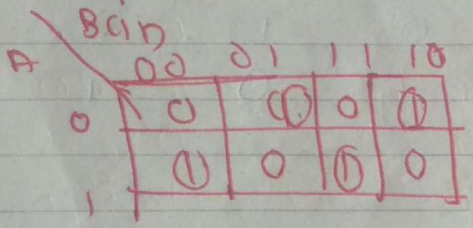


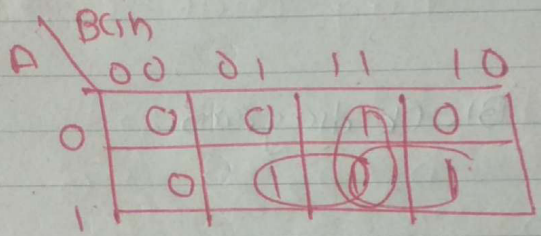
a) Block diagram.

I/P			O/P	
A	B	Cin	S	Co
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

K-map

For sum





$S = A \oplus B \oplus cin$

$C = A \cdot cin + B \cdot cin + A \cdot B$

Logic diagram :-

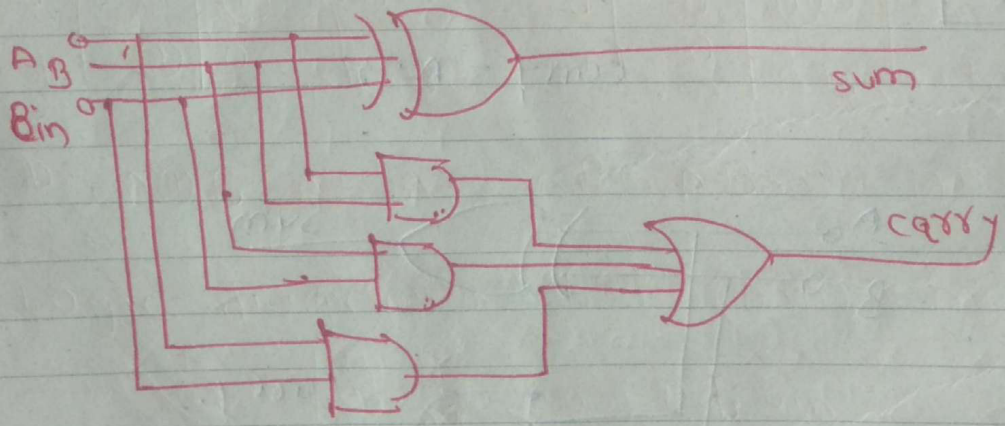
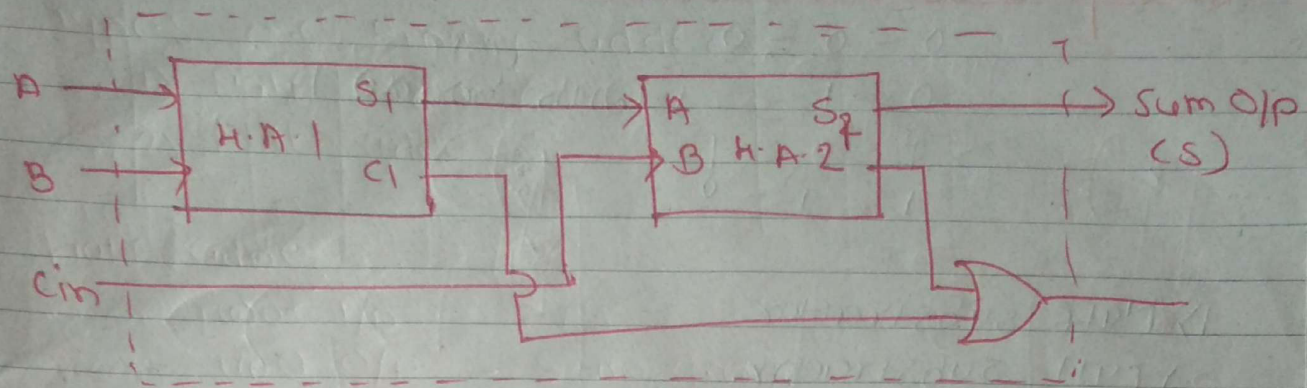


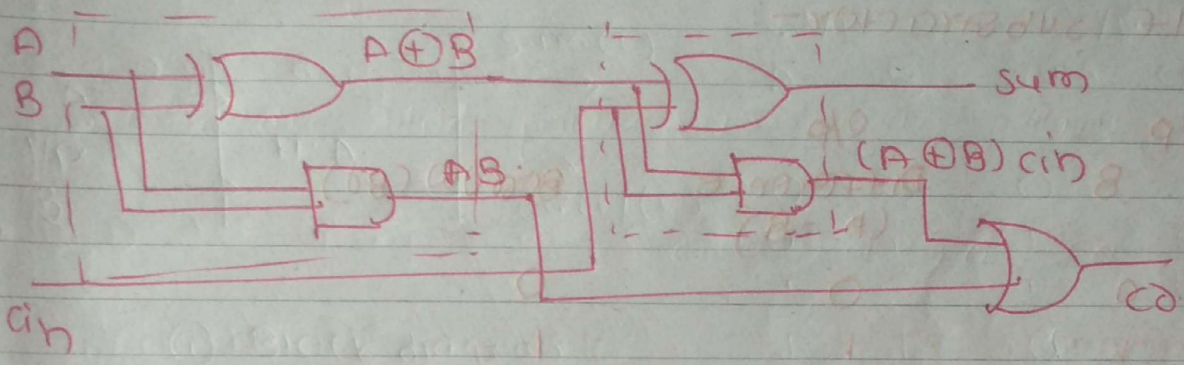
Fig :- Full-adder ckt.

https://t.me/SPPU_TE_BE_COMP

Scanned by CamScanner

Full adder using Half adder :-





Proof

$S = A \oplus B \oplus C$

Carry $C = AB + (A \oplus B) C_{in}$

$= AB + \bar{A}BC_{in} + A\bar{B}C_{in}$

$= AB(1 + C_{in}) + \bar{A}BC_{in} + A\bar{B}C_{in}$

$= AB + ABC_{in} + \bar{A}BC_{in} + A\bar{B}C_{in}$

$= BC_{in}(A + \bar{A}) + AB + A\bar{B}C_{in}$

$= BC_{in} + AB(1 + C_{in}) + A\bar{B}C_{in}$

$= BC_{in} + AB + ABC_{in} + A\bar{B}C_{in}$

$= BC_{in} + AC_{in}(B + \bar{B}) + AB$

$$C = BC_{in} + AC_{in} + AB$$

https://t.me/SPPU_TE_BE_COMP

Scanned by CamScanner

Binary Substractor –:

0 – 0 = 0

0 – 1 = 1 with borrow 1.

1 – 0 = 1

1 – 1 = 0

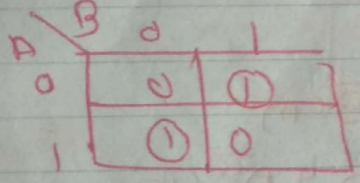
1) Half substractor.
2) Full substractor.

Half substractor –:

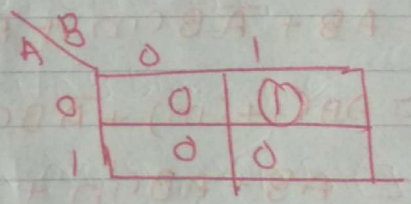
I/p A	I/p B	O/p Difference (A – B)	O/p Borrow (BO)
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

K-map –:

Difference.

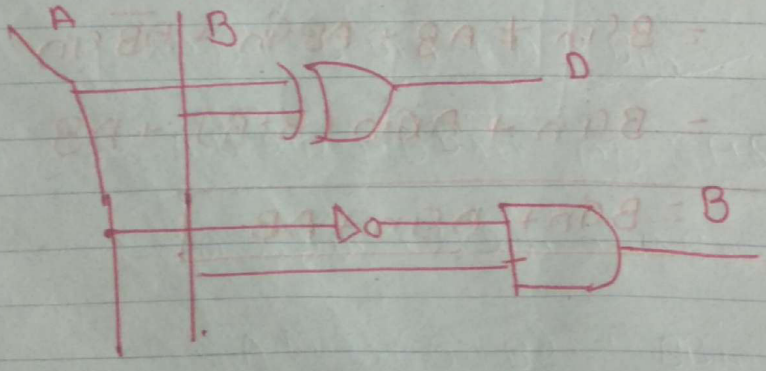


Borrow



$D = A \oplus B$ $B = \overline{A}B$

Logic diagram –



Half substractor ckt.

https://t.me/SPPU_TE_BE_COMP

Scanned by CamScanner

Full Substractor -

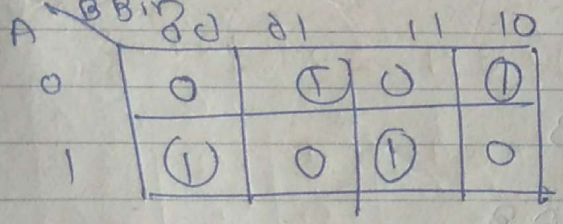
I/P - A, B, Bin (previous borrow)

O/P = (A - B - Bin), BO

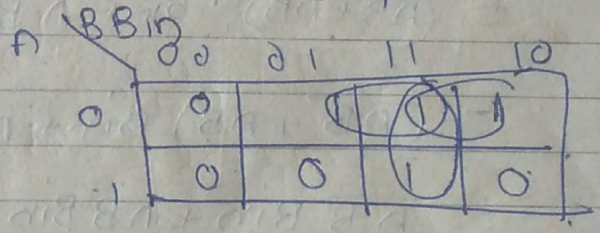
A	B	Bin	O/P (A-B-Bin)	BO
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

K-map :-

For difference



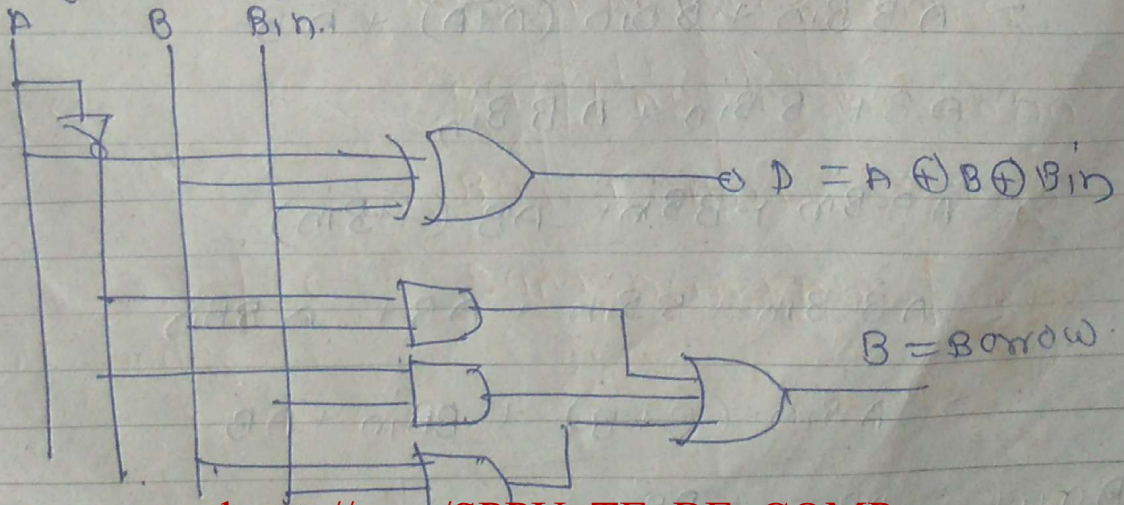
For carry.



$D = A \oplus B \oplus Bin$

$Bo = \overline{A} \cdot Bin + \overline{A} \cdot B + B\,Bin$

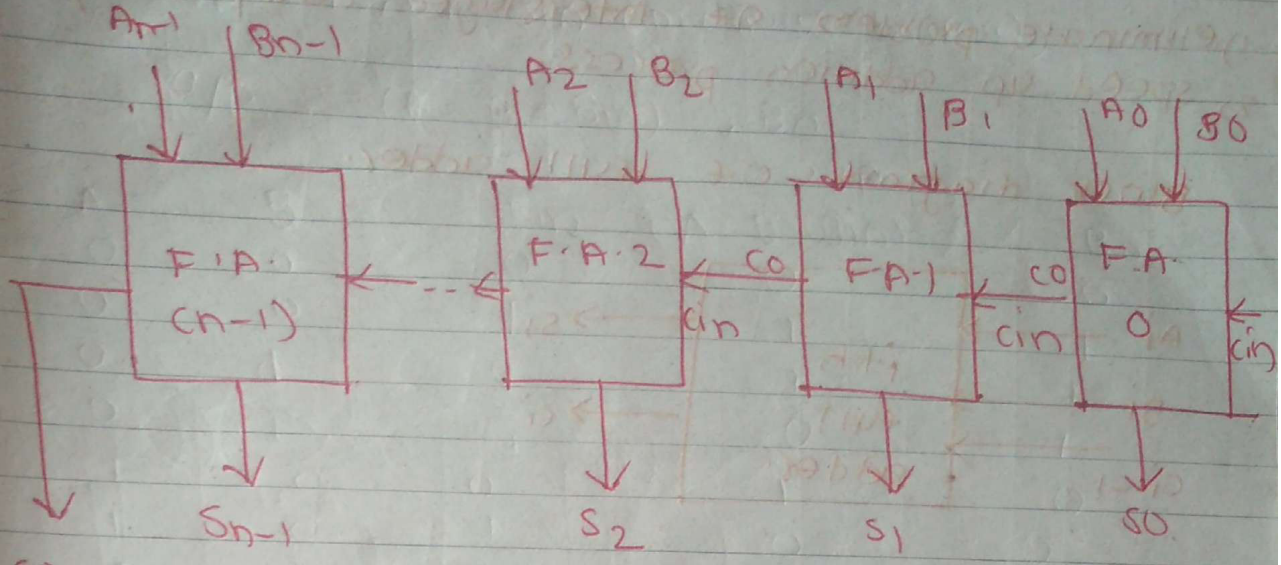
Logic diagram of full substractor



https://t.me/SPPU_TE_BE_COMP

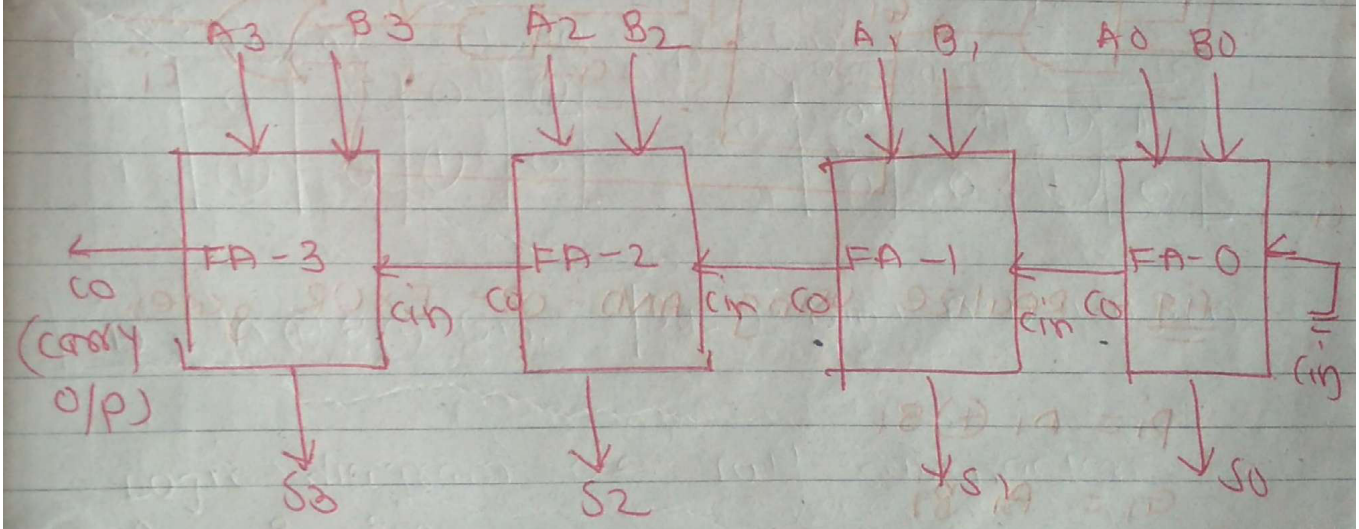
Scanned by CamScanner

n-bit parallel adder :-

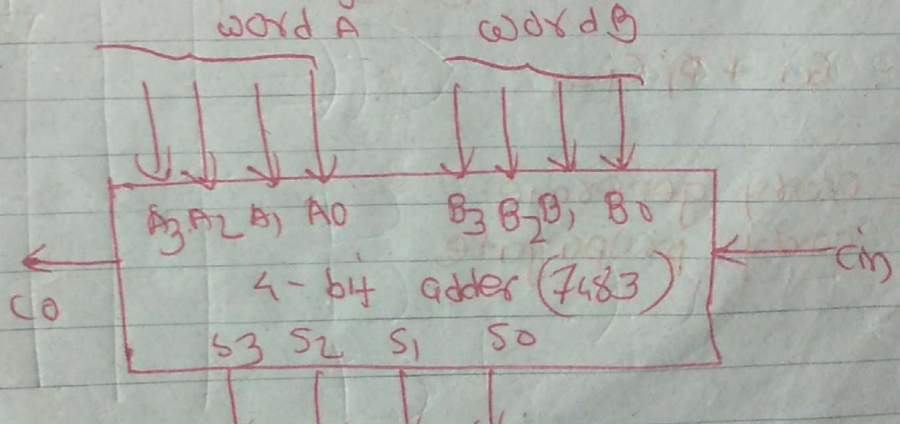


Block diagram of n-bit ||el adder.

* Four bit ||el adder using full adder :-



Block diagram of four bit ||el adder.



https://t.me/SPPU_TE_BE_COMP
Scanned by CamScanner

Design a ckt. sum is greater than 9 & carry = 1
The o/p of combinational should be 1 if sum produced by adder-1 greater than 9. i.e. 1001.

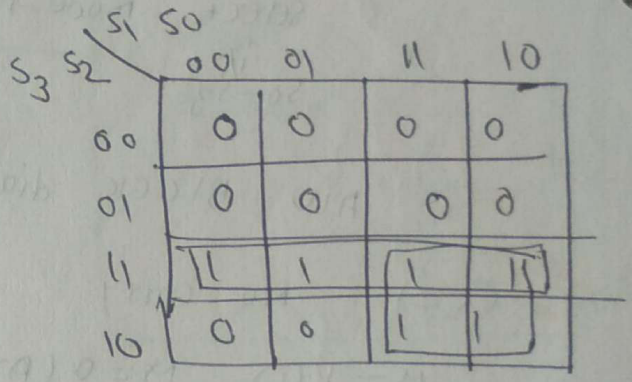
* BCD adder :-

Truth table for combinational ckt. :-

I/p				o/p
S_3	S_2	S_1	S_0	Y
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	0
1	0	0	0	0
1	0	0	1	0
1	0	1	0	1
1	0	1	1	1
1	1	0	0	1
1	1	0	1	1
1	1	1	0	1
1	1	1	1	1

Sum is valid BCD no.

$\therefore Y = 0$



$Y = S_3 \cdot S_2 + S_3 \cdot S_1$

Sum is invalid BCD NO

$Y = 1$

Que:- Design 4-bit BCD adder using binary adder IC's

OR

Design 1-digit BCD adder using IC 74LS83

https://t.me/SPPU_TE_BE_COMP

Scanned by CamScanner

I) sum ≤ 9 & carry = 0 (IC 7483)

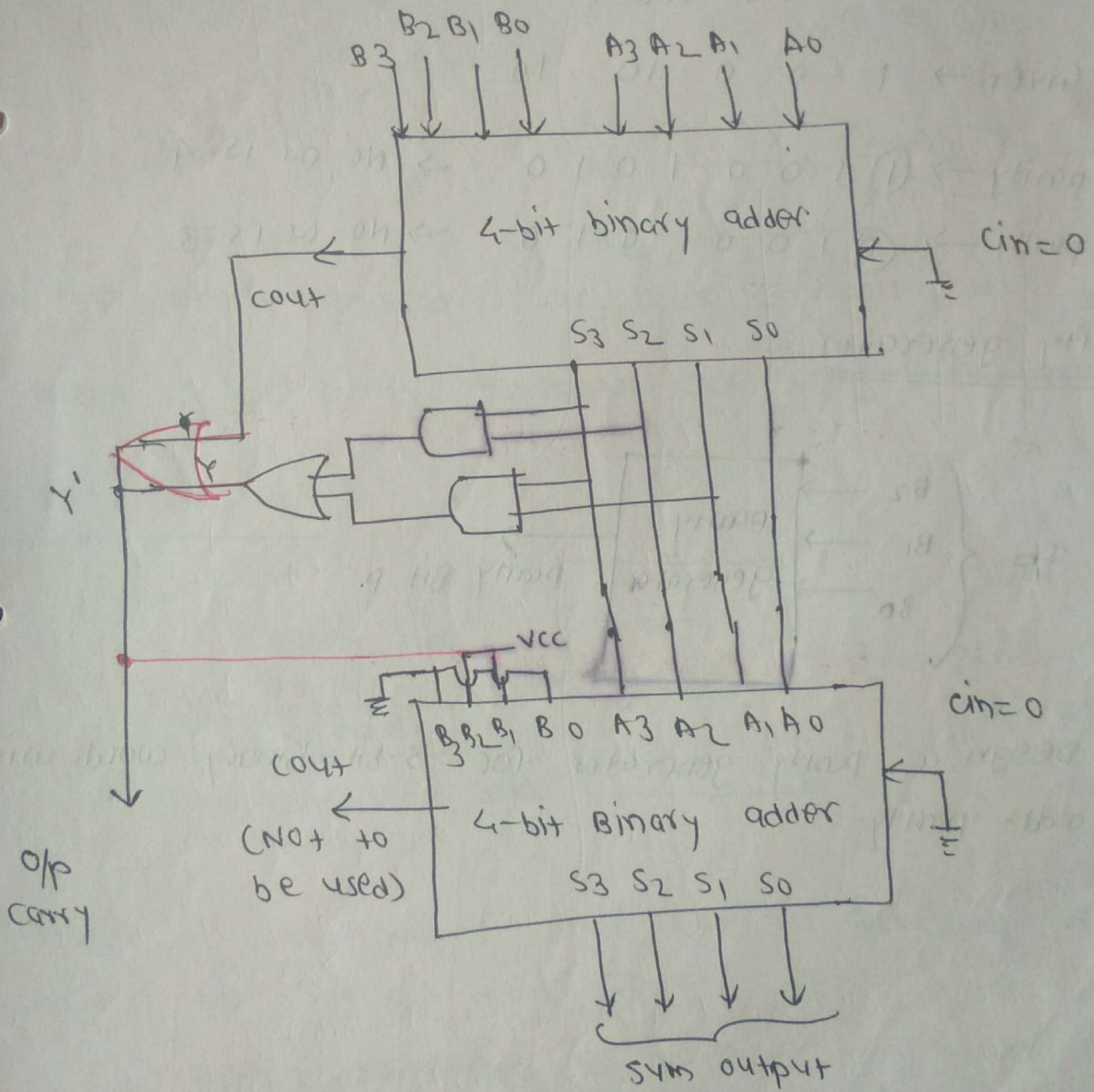
Y' = 0 hence $B_3B_2B_1B_0 = 0000$ $\overset{5}{+4}$

II) sum > 9 & carry = 0

Y' = 1 , $B_3B_2B_1B_0 = 0110$ then will get correct BCD No. $\overset{5}{+5}$

III) sum ≤ 9 but carry = 1 $\overset{9}{+9}$

Y' = 1 , add - 6



4-bit BCD adder

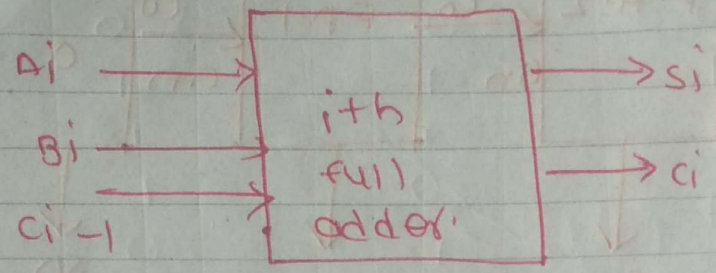
https://t.me/SPPU_TE_BE_COMP

Scanned by CamScanner

Look ahead & carry adder -1

1) eliminate problem of interstage carry delay.
2) speed up addition process.

Block diagram of full adder.



Block diagram of ith full adder.

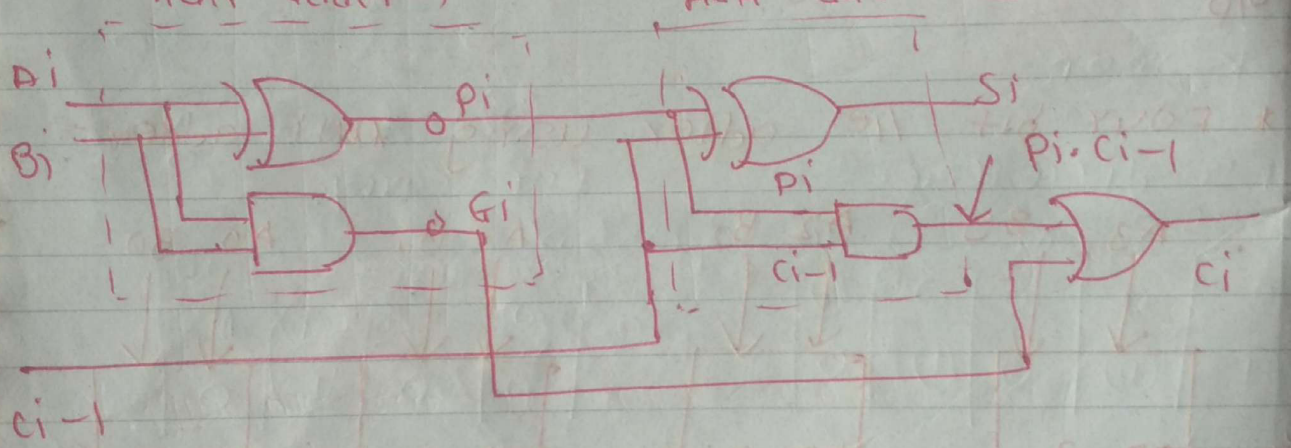


fig - Realize using AND-OR-EXOR gates.

$P_i = A_i \oplus B_i$

$G_i = A_i B_i$

$S_i = P_i \oplus C_{i-1} = A \oplus B \oplus C_{i-1}$

$C_i = G_i + P_i C_{i-1}$

G_i = carry generate
P_i = carry propagate

https://t.me/SPPU_TE_BE_COMP

Scanned by CamScanner

$C_i = G_i + P_i C_{i-1}$

Stage	expression for carry o/p
0	$C_0 = G_0 + P_0 C_{-1}$
1	$C_1 = G_1 + P_1 C_0$
	$= G_1 + P_1 (G_0 + P_0 C_{-1})$
	$C_1 = G_1 + P_1 G_0 + P_1 P_0 C_{-1}$
2	$C_2 = G_2 + P_2 C_1$
	$= G_2 + P_2 (G_1 + P_1 C_0)$
	$C_2 = G_2 + G_1 P_2 + P_1 P_2 C_0$
3	$C_3 = G_2 + G_1 P_2 + P_1 P_2 (G_0 + P_0 C_{-1})$
	$= G_2 + G_1 P_2 + P_1 P_2 G_0 + P_0 P_1 P_2 C_{-1}$
3	$C_3 = G_3 + P_3 C_2$
	$= G_3 + P_3 (G_2 + P_2 C_1)$
	$= G_3 + P_3 G_2 + P_3 P_2 C_1$
	$= G_3 + P_3 G_2 + P_3 P_2 (G_1 + P_1 G_0 + P_1 P_0$
	$= G_3 + P_3 G_2 + P_3 P_2 G_1 + P_1 P_2 P_3 G_0$
	$+ P_0 P_1 P_2 P_3 C_{-1}$

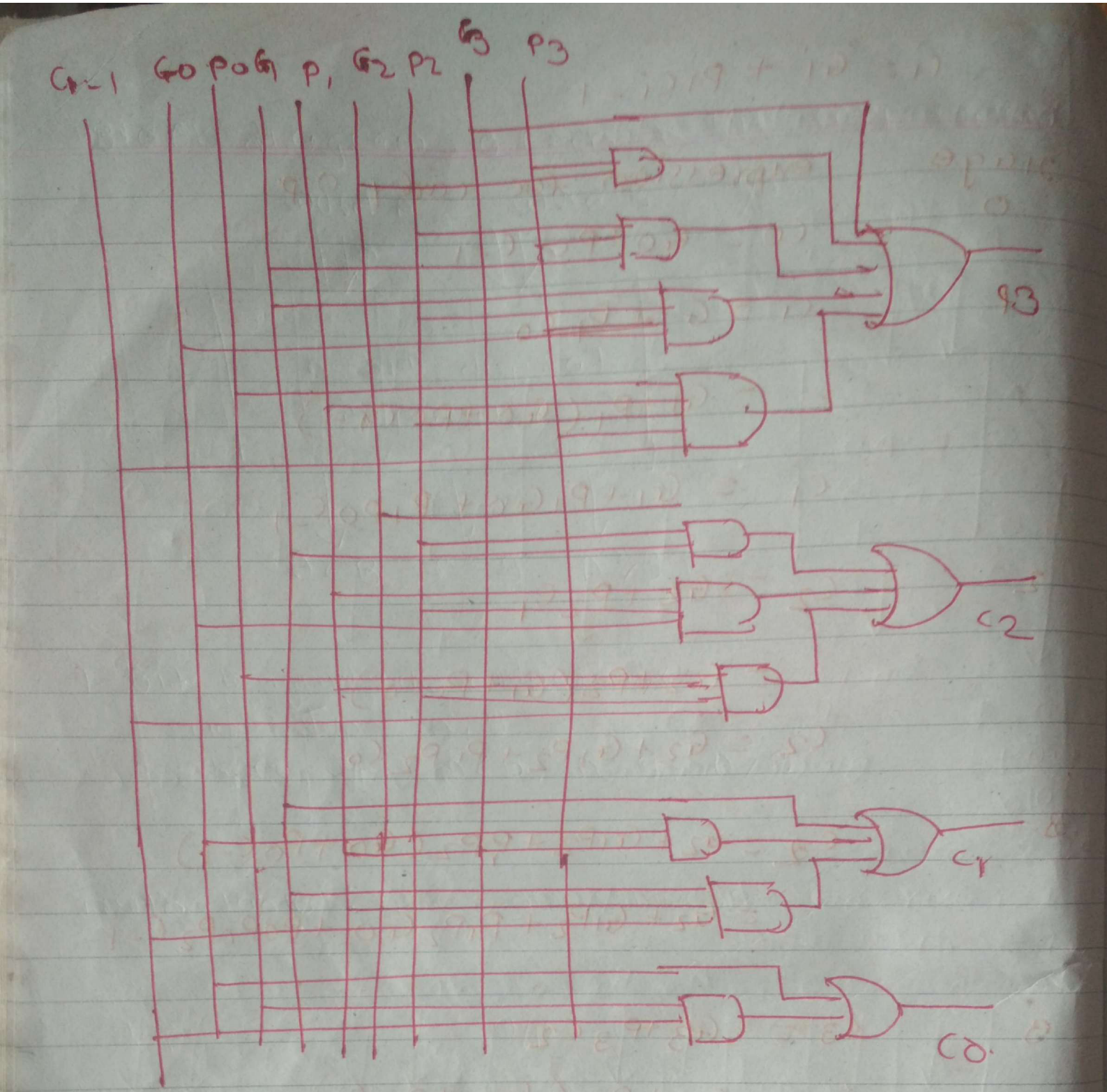
expression involved $G_0, G_1, G_2, G_3, P_0, P_1, P_2, P_3$ & C_{-1}

G_i = carry generate
P_i = carry propagate

G_i value do not depends on C_{i-1} (I/p carry)
P_i term associated with the carry propagate

https://t.me/SPPU_TE_BE_COMP

Scanned by CamScanner



Logic diagram of look ahead carry generator.

Four bit Fast adder with look-ahead carry -:

https://t.me/SPPU_TE_BE_COMP

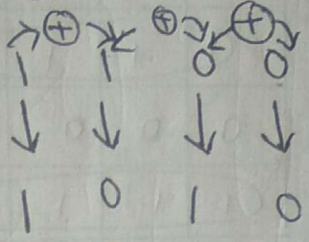
Scanned by CamScanner

de converters :-

Binary to gray code converter :- ④

Decimal	Binary I/p				Gray o/p			
	B_3	B_2	B_1	B_0	G_3	G_2	G_1	G_0
0	0	0	0	0	0	0	0	0
1	0	0	0	1	0	0	0	1
2	0	0	1	0	0	0	1	1
3	0	0	1	1	0	0	1	0
4	0	1	0	0	0	1	1	0
5	0	1	0	1	0	1	1	1
6	0	1	1	0	0	1	0	1
7	0	1	1	1	0	1	0	0
8	1	0	0	0	1	1	0	0
9	1	0	0	1	1	1	0	1
10	1	0	1	0	1	1	1	1
11	1	0	1	1	1	1	1	0
12	1	1	0	0	1	0	1	0
13	1	1	0	1	1	0	1	1
14	1	1	1	0	1	0	0	1
15	1	1	1	1	1	0	0	0

e.g



Truth table EX-OR

A	B	EX-OR
0	0	0
0	1	1
1	0	1
1	1	0

K-map :-

FOR G_0

B_3B_2 \ B_1B_0	00	01	11	10
00	0	1	0	1
01	0	1	0	1
11	0	1	0	1
10	0	1	0	1

$G_0 = \overline{B_1}B_0 + B_1\overline{B_0} = B_1 \oplus B_0$

FOR G_1

B_3B_2 \ B_1B_0	00	01	11	10
00	0	0	1	1
01	1	1	0	0
11	1	1	0	0
10	0	0	1	1

$G_1 = B_2\overline{B_1} + \overline{B_2}B_1 = B_1 \oplus B_2$

https://t.me/SPPU_TE_BE_COMP

Scanned by CamScanner

For G_2

B_3B_2 \ B_1B_0	00	01	11	10
00	0	0	0	0
01	1	1	1	1
11	0	0	0	0
10	1	1	1	1

$G_2 = \overline{B_3} B_2 + B_3 \overline{B_2}$

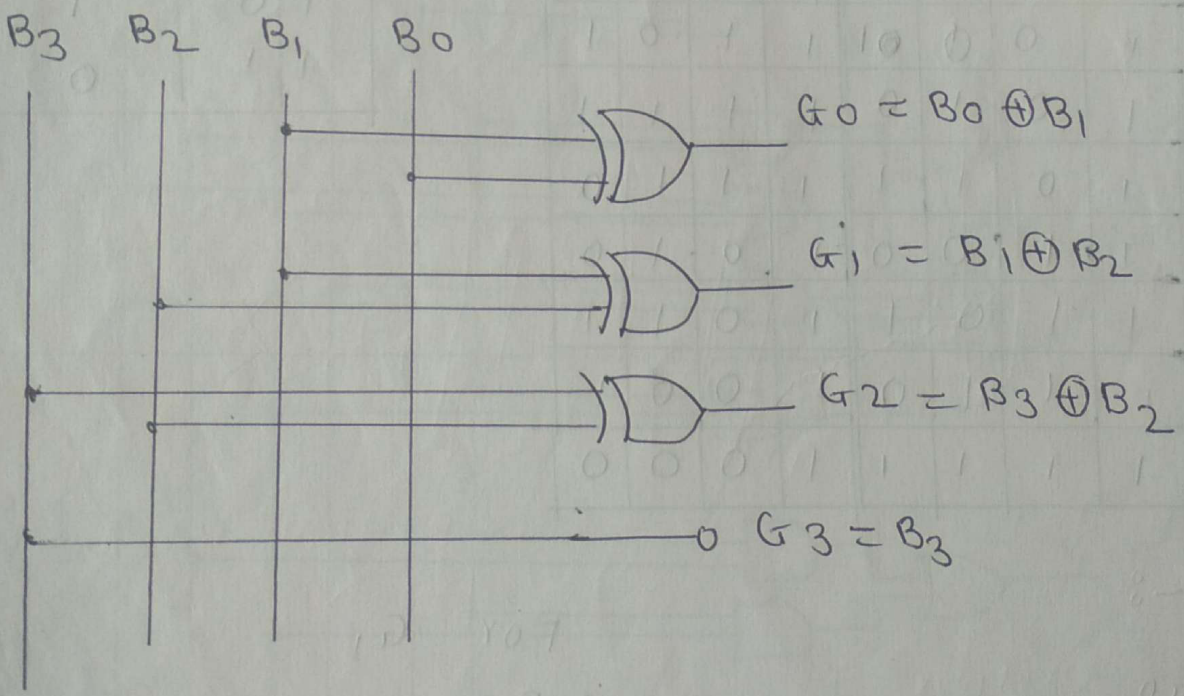
$G_2 = B_3 \oplus B_2$

For G_3

B_3B_2 \ B_1B_0	00	01	11	10
00	0	0	0	0
01	0	0	0	0
11	1	1	1	1
10	1	1	1	1

$G_3 = B_3$

Logic diagram :-



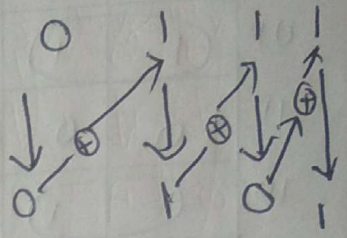
https://t.me/SPPU_TE_BE_COMP

Scanned by CamScanner

Gray to Binary :-

Decimal	I/p				O/p			
	G_3	G_2	G_1	G_0	B_3	B_2	B_1	B_0
0	0	0	0	0	0	0	0	0
1	0	0	0	1	0	0	0	1
3	0	0	1	1	0	0	1	0
2	0	0	1	0	0	0	1	1
6	0	1	1	0	0	1	0	0
7	0	1	1	1	0	1	0	1
5	0	1	0	1	0	1	1	0
4	0	1	0	0	0	1	1	1
12	1	1	0	0	1	0	0	0
13	1	1	0	1	1	0	0	1
15	1	1	1	1	1	0	1	0
14	1	1	1	0	1	0	1	1
10	1	0	1	0	1	1	0	0
11	1	0	1	1	1	1	0	1
9	1	0	0	1	1	1	1	0
8	1	0	0	0	1	1	1	1

e.g



K-map :-

FOR B_3

G_3G_2 \ G_1G_0	00	01	11	10
00	0	0	0	0
01	0	0	0	0
11	1	1	1	1
10	1	1	1	1

$B_3 = G_3$

FOR B_2

G_3G_2 \ G_1G_0	00	01	11	10
00	0	0	0	0
01	1	1	1	1
11	0	0	0	0
10	1	1	1	1

$B_2 = G_2 \oplus G_3$

https://t.me/SPPU_TE_BE_COMP

Scanned by CamScanner

For B_1

G_3G_2 \ G_1G_0	00	01	11	10
00	0	0	1	1
01	1	1	0	0
11	0	0	1	1
10	1	1	0	0

$$B_1 = \overline{G_1}\,\overline{G_3}\cdot G_2 + \overline{G_3}\,\overline{G_2}\cdot G_1 + G_3G_2\cdot G_1 + G_3\cdot\overline{G_2}\cdot\overline{G_1}$$

$$= G_1(\overline{G_3}\cdot\overline{G_2} + G_3G_2) + \overline{G_1}(\overline{G_3}\cdot G_2 + \overline{G_2}\cdot G_3)$$

$$B_1 = G_1 \oplus G_2 \oplus G_3$$

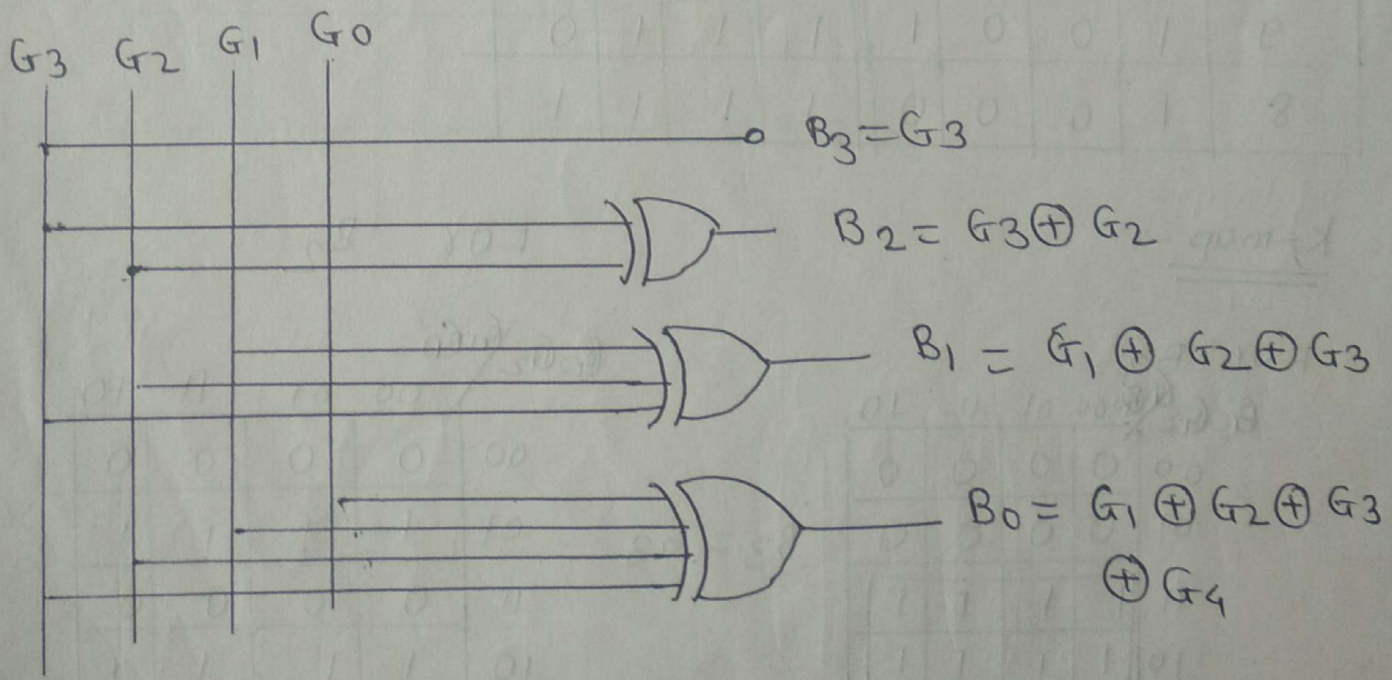
For B_0

G_3G_2 \ G_1G_0	00	01	11	10
00	0	1	0	1
01	1	0	1	0
11	0	1	0	1
10	1	0	1	0

$$B_0 = \overline{G_3}\cdot\overline{G_2}\cdot\overline{G_1}\cdot G_0 + \overline{G_3}\,\overline{G_2}\,G_1\,\overline{G_0} + \overline{G_1}\,G_0\,\overline{G_3}\,G_2 + \overline{G_1}\,G_0\,\overline{G_3}\,G_2 + G_3G_2\overline{G_1}\,G_0 + G_3G_2G_1\cdot\overline{G_0} + \overline{G_1}\,\overline{G_0}\,G_3\,\overline{G_2} + G_3\overline{G_2}G_1G_0$$

$$B_0 = G_1 \oplus G_2 \oplus G_3 \oplus G_4$$

Logic diagram :-



https://t.me/SPPU_TE_BE_COMP

Scanned by CamScanner

CD to Excess-3

Decimal	~~Binary~~ BCD				Excess-3			
	B_3	B_2	B_1	B_0	E_3	E_2	E_1	E_0
0	0	0	0	0	0	0	1	1
1	0	0	0	1	0	1	0	0
2	0	0	1	0	0	1	0	1
3	0	0	1	1	0	1	1	0
4	0	1	0	0	0	1	1	1
5	0	1	0	1	1	0	0	0
6	0	1	1	0	1	0	0	1
7	0	1	1	1	1	0	1	0
8	1	0	0	0	1	0	1	1
9	1	0	0	1	1	1	0	0
10	1	0	1	0	X	X	X	X
11	1	0	1	1	X	X	X	X
12	1	1	0	0	X	X	X	X
13	1	1	0	1	X	X	X	X
14	1	1	1	0	X	X	X	X
15	1	1	1	1	X	X	X	X

* K-map:-

FOR E_0

B_3B_2 \ B_1B_0	00	01	11	10
00	1	0	0	1
01	1	0	0	1
11	X	X	X	X
10	1	0	X	X

$E_0 = \overline{B_0}$

FOR E_1

B_3B_2 \ B_1B_0	00	01	11	10
00	1	0	1	0
01	1	0	1	0
11	X	X	X	X
10	1	0	X	X

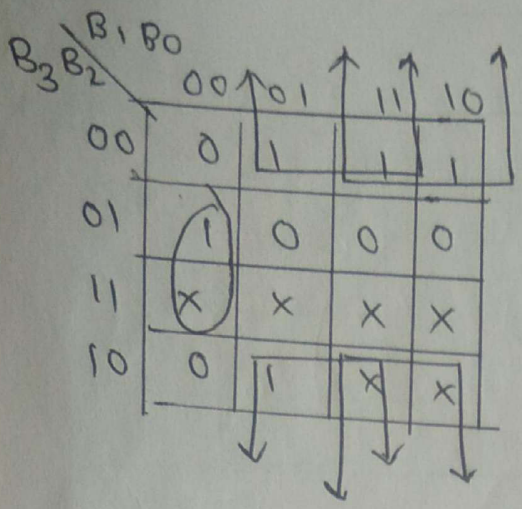
$E_1 = B_1B_0 + \overline{B_1}\,\overline{B_0}$

$= \overline{B_1 \oplus B_0}$

https://t.me/SPPU_TE_BE_COMP

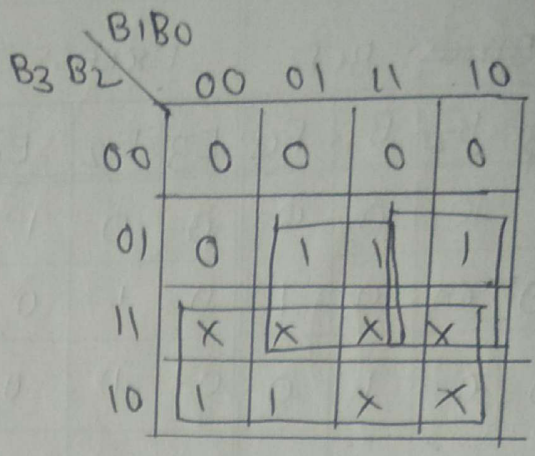
Scanned by CamScanner

For E_2



$$E_2 = B_2\bar{B}_1\bar{B}_0 + \bar{B}_2B_0 + \bar{B}_2B_1$$

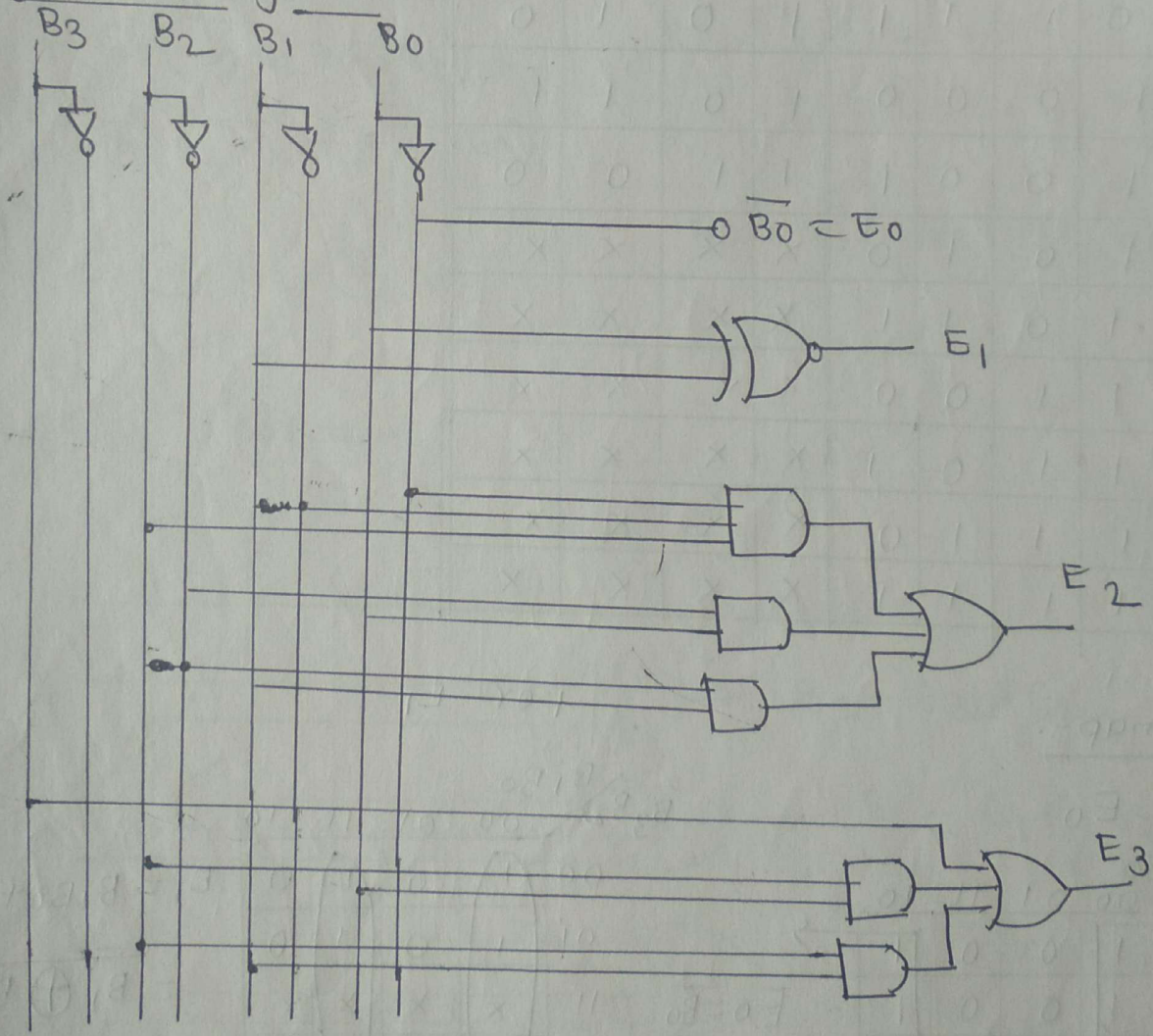
For E_3



$$E_3 = B_3 + B_2B_0 + B_2B_1$$

$$E_3 = B_3 + B_2(B_1 + B_0)$$

Logic diagram :-



https://t.me/SPPU_TE_BE_COMP

Scanned by CamScanner

cess-3 to BCD :-

with table :-

decimal	Excess-3				BCD			
	E_3	E_2	E_1	E_0	D_3	D_2	D_1	D_0
0	0	0	0	0	x	x	x	x
1	0	0	0	1	x	x	x	x
2	0	0	1	0	x	x	x	x
3	0	0	1	1	0	0	0	0
4	0	1	0	0	0	0	0	1
5	0	1	0	1	0	0	1	0
6	0	1	1	0	0	0	1	1
7	0	1	1	1	0	1	0	0
8	1	0	0	0	0	1	0	1
9	1	0	0	1	0	1	1	0
10	1	0	1	0	0	1	1	1
11	1	0	1	1	1	0	0	0
12	1	1	0	0	1	0	0	1
13	1	1	0	1	x	x	x	x
14	1	1	1	0	x	x	x	x
15	1	1	1	1	x	x	x	x

K-map :-

For D_0

E_3E_2 \ E_1E_0	00	01	11	10
00	x	x	0	x
01	1	0	0	1
11	1	x	x	x
10	1	0	0	1

$D_0 = \overline{E_0}$

For D_1

E_3E_2 \ E_1E_0	00	01	11	10
00	x	x	0	x
01	0	1	0	1
11	0	x	x	x
10	0	1	0	1

$D_1 = \overline{E_1}E_0 + E_1\overline{E_0}$

$D_1 = E_1 \oplus E_0$

https://t.me/SPPU_TE_BE_COMP

Scanned by CamScanner

For D_2

E_3E_2 \ E_1E_0	00	01	11	10
00	X	X	0	X
01	0	0	1	0
11	0	X	X	X
10	1	1	0	1

$D_2 = \overline{E_2} \cdot \overline{E_1} + E_2 E_1 E_0 + E_3 E_1 \overline{E_0}$

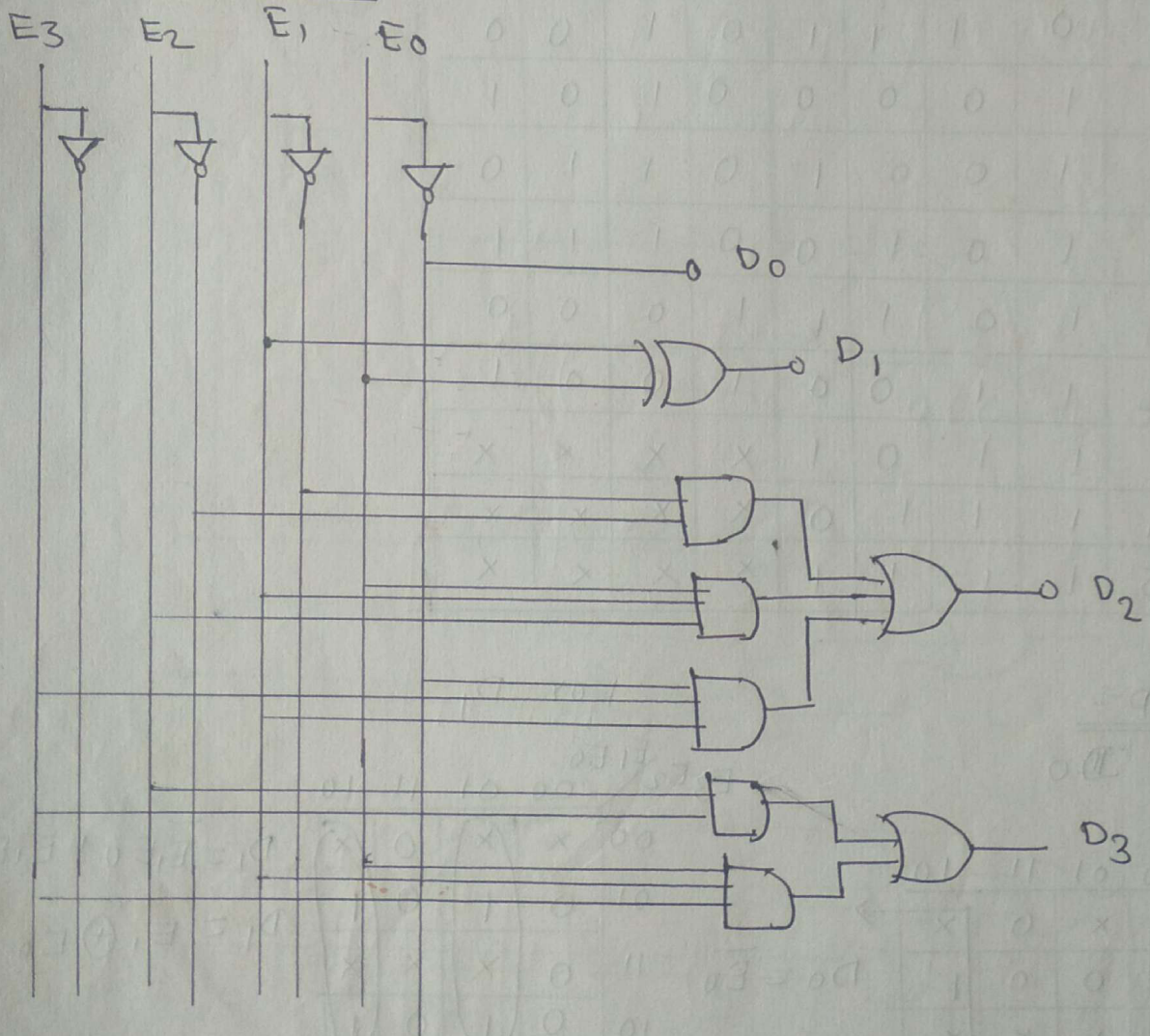
For D_3

E_3E_2 \ E_1E_0	00	01	11	10
00	X	X	0	X
01	0	0	0	0
11	1	X	X	X
10	0	0	1	0

$D_3 = E_3 \cdot E_2 + E_1 E_0 E_3$

$D_3 = E_3 (E_2 + E_1 E_0)$

Logic diagram :-



https://t.me/SPPU_TE_BE_COMP

Scanned by CamScanner

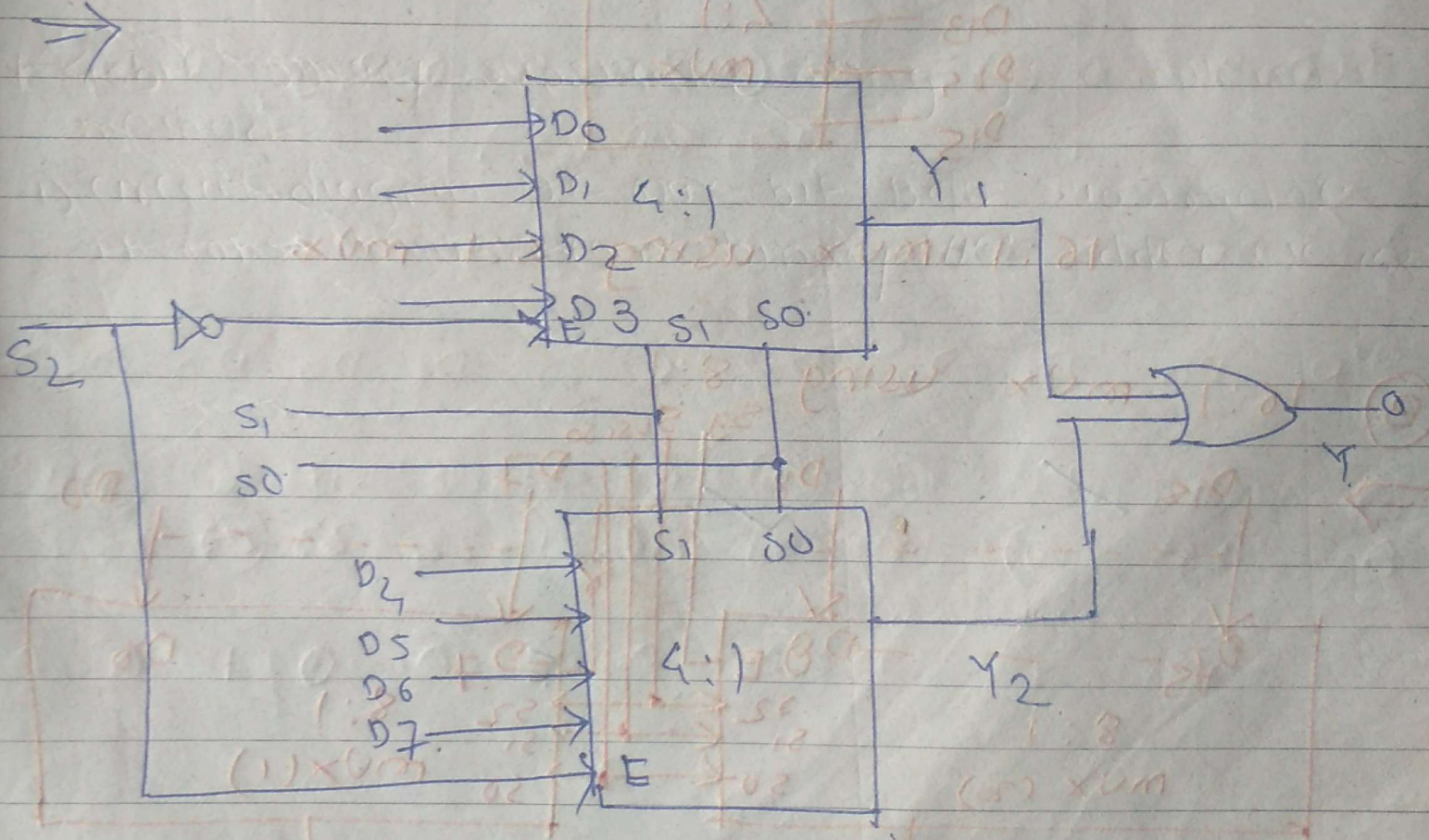
* Multiplexer :-

Types of multiplexer

1) 2:1
2) 4:1
3) 8:1
4) 16:1
5) 32:1

Multiplexer tree :- cascading two or more multiplexer with less no. of i/p.

Q. 1) Obtain 8:1 mux using 4:1



8:1 mux by cascading two 4:1 mux

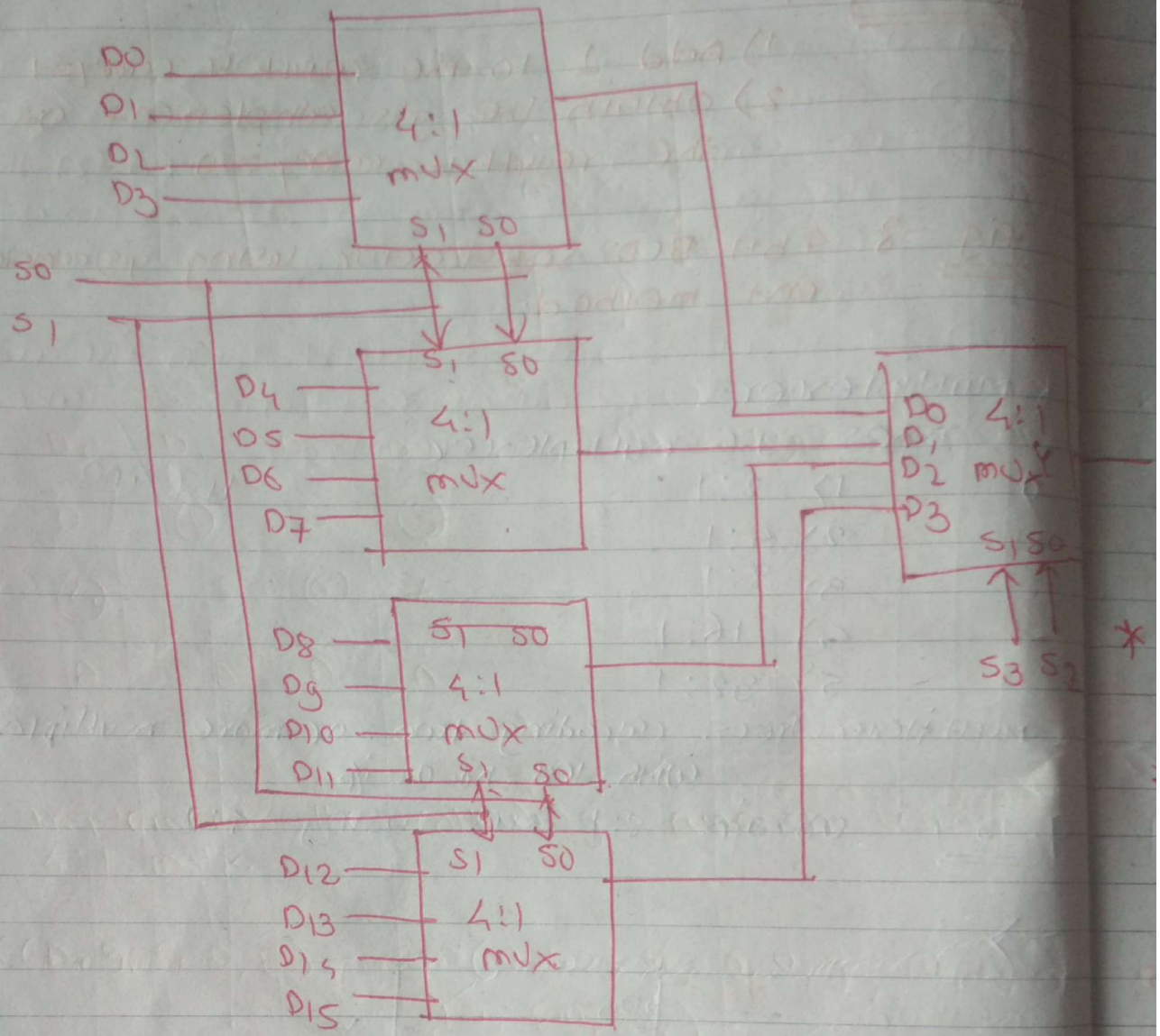
https://t.me/SPPU_TE_BE_COMP

Scanned by CamScanner

② Implement 16:1 mux using 4:1 mux.

⇒

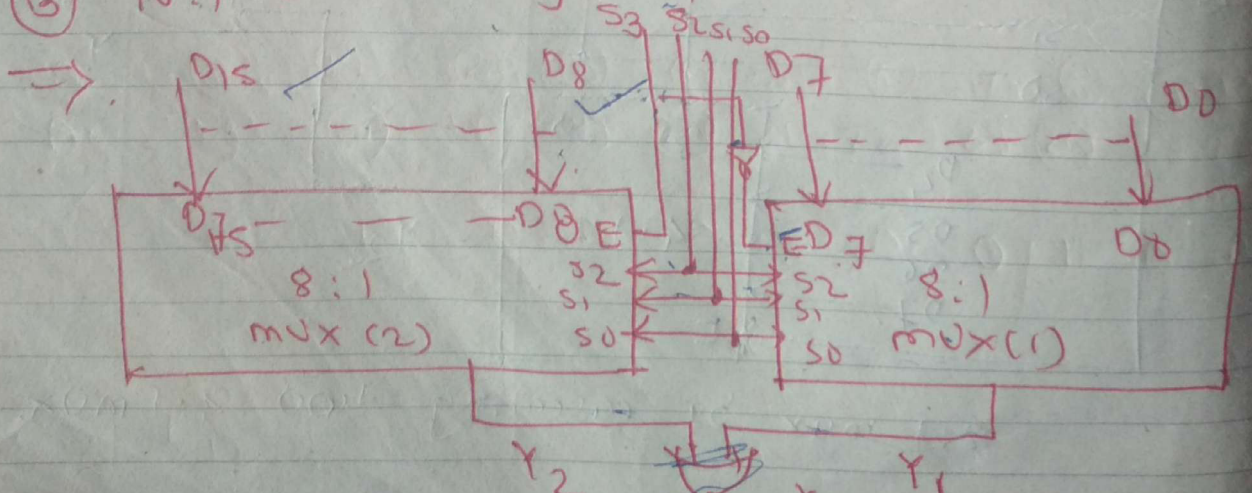
00
01
10
11



16:1 mux using 4:1 mux.

③ 16:1 mux using 8:1

⇒

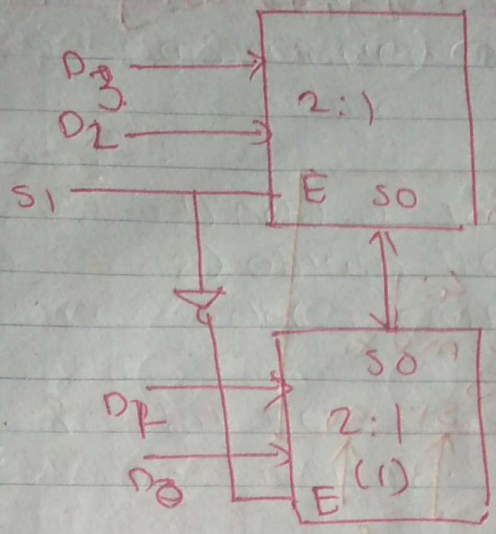


https://t.me/SPPU_TE_BE_COMP

Scanned by CamScanner

* 4:1 using 2:1

⇒



4:1 mux using 2:1 mux.

— o/p.

* 32:1 using 16:1 mux.

* Implement combinational logic using mux.

7.4.2 Implement the following expression using 8:1 mux.

$$f(A,B,C) = \Sigma m(0,2,4,6)$$

⇒

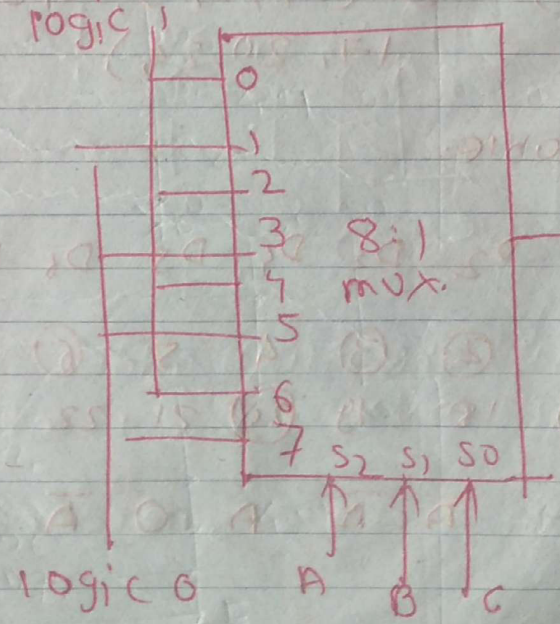


fig :- Implement the logic expression

https://t.me/SPPU_TE_BE_COMP

Scanned by CamScanner

③ Implement the logic function

$f(A,B,C) = \Sigma m(1,3,4,6)$ using 4:1 MUX

⇒

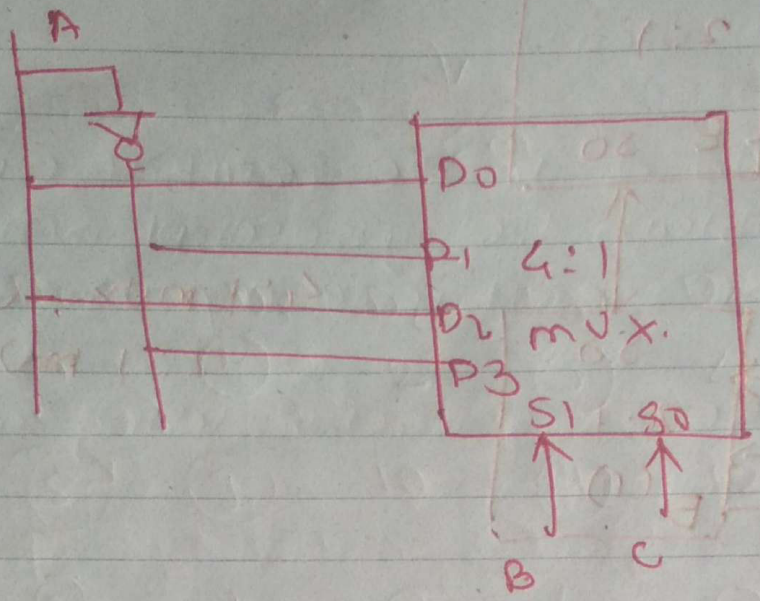


fig :- logic diagram

	D_0	D_1	D_2	D_3
$\bar{A}$	0	1	2	3
A	4	5	6	7
	A	$\bar{A}$	A	$\bar{A}$

https://t.me/SPPU_TE_BE_COMP

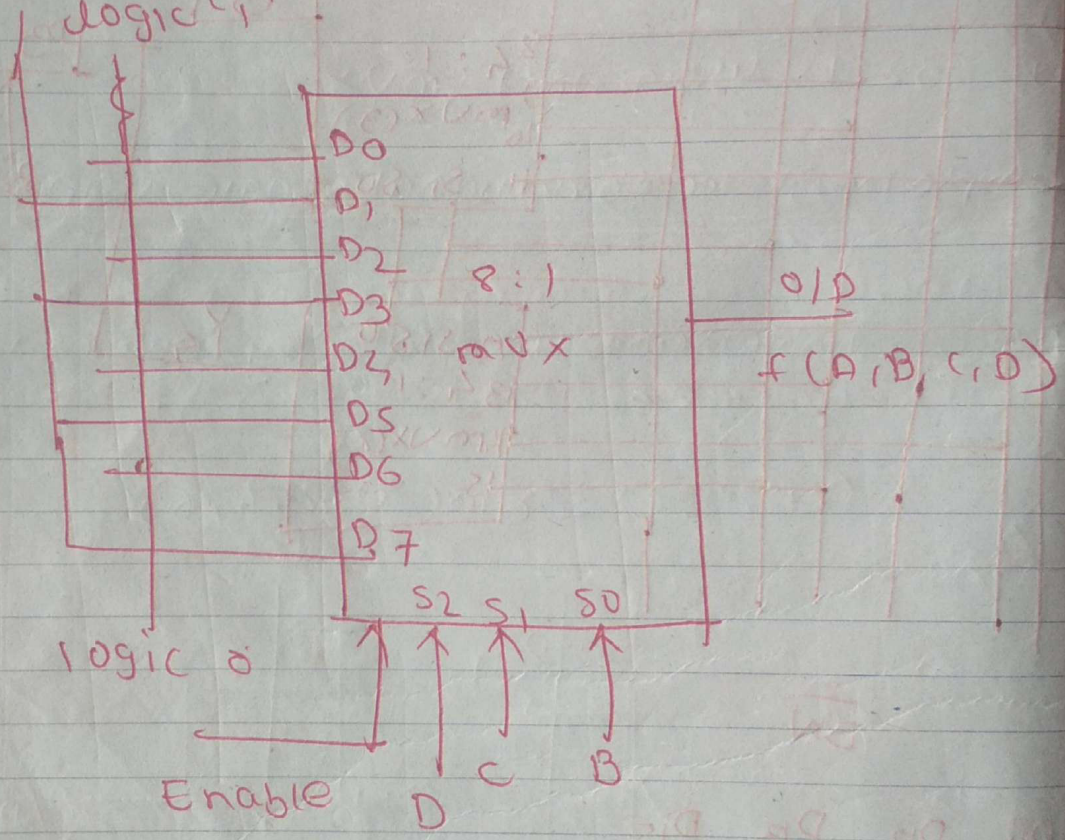
Scanned by CamScanner

⑤ Implement the following function using 8:1 mux.

⇒ $f(A,B,C,D) = \sum m(0,2,4,6,8,10,12,14)$

⇒ $= \Sigma m$

	D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7
$\bar{A}$	0	(1)	2	(3)	4	(5)	6	(7)
A	(8)	(9)	10	(11)	12	(13)	14	(15)
	A	1	0	1	0	1	0	1



⑫ Implement the full adder using two 4:1 mux. & 8:1 mux.

⇒

A	B	C	sum	carry
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0

https://t.me/SPPU_TE_BE_COMP

Scanned by CamScanner

			Sum	carry.
0	1	1	0	1
1	0	0	1	0
0	0	1	0	1
1	1	0	0	1
1	1	1	1	1

$f(A,B,C) = \Sigma m(1,2,4,7) \rightarrow$ sum

carry $= f(A,B,C) = \Sigma m(3,5,6,7)$

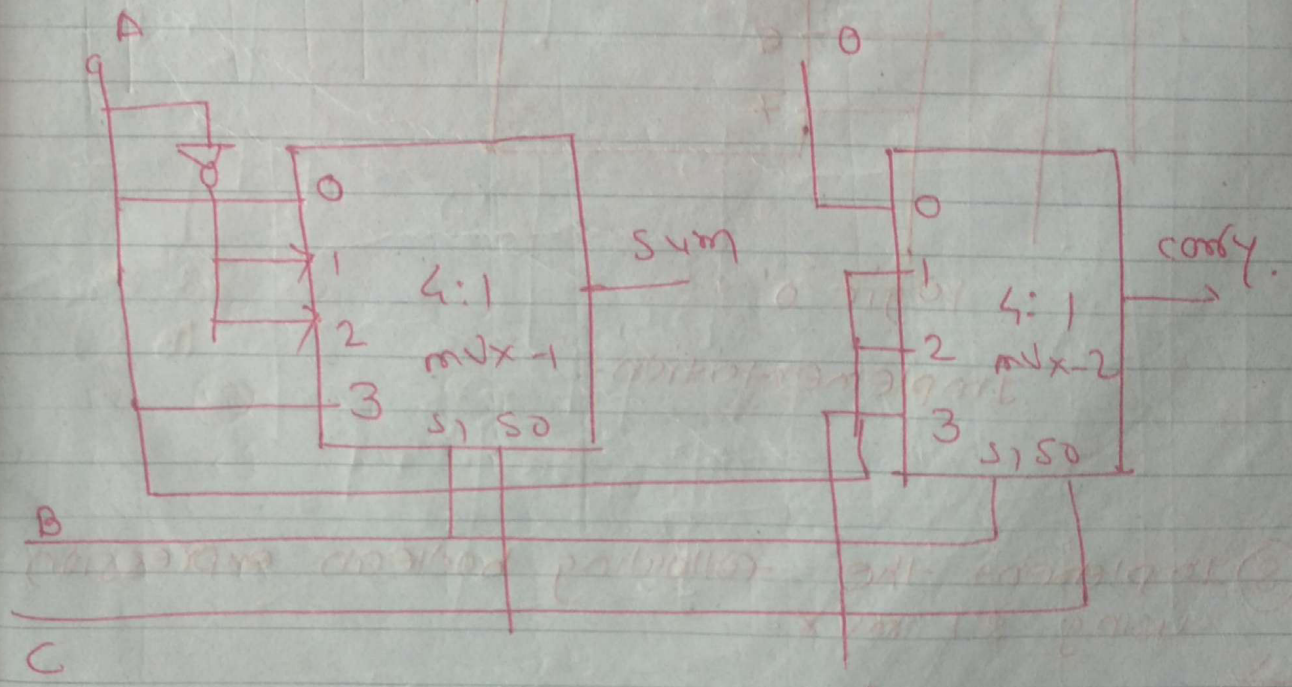
Design table

Sum

	D_0	D_1	D_2	D_3
$\bar{A}$	0	(1)	(2)	3
A	(4)	5	6	(7)
	A	$\bar{A}$	$\bar{A}$	A

carry.

	D_0	D_1	D_2	D_3
$\bar{A}$	0	1	2	(3)
A	4	(5)	(6)	(7)
	0	A	A	1



full adder using 4:1 mux.

https://t.me/SPPU_TE_BE_COMP

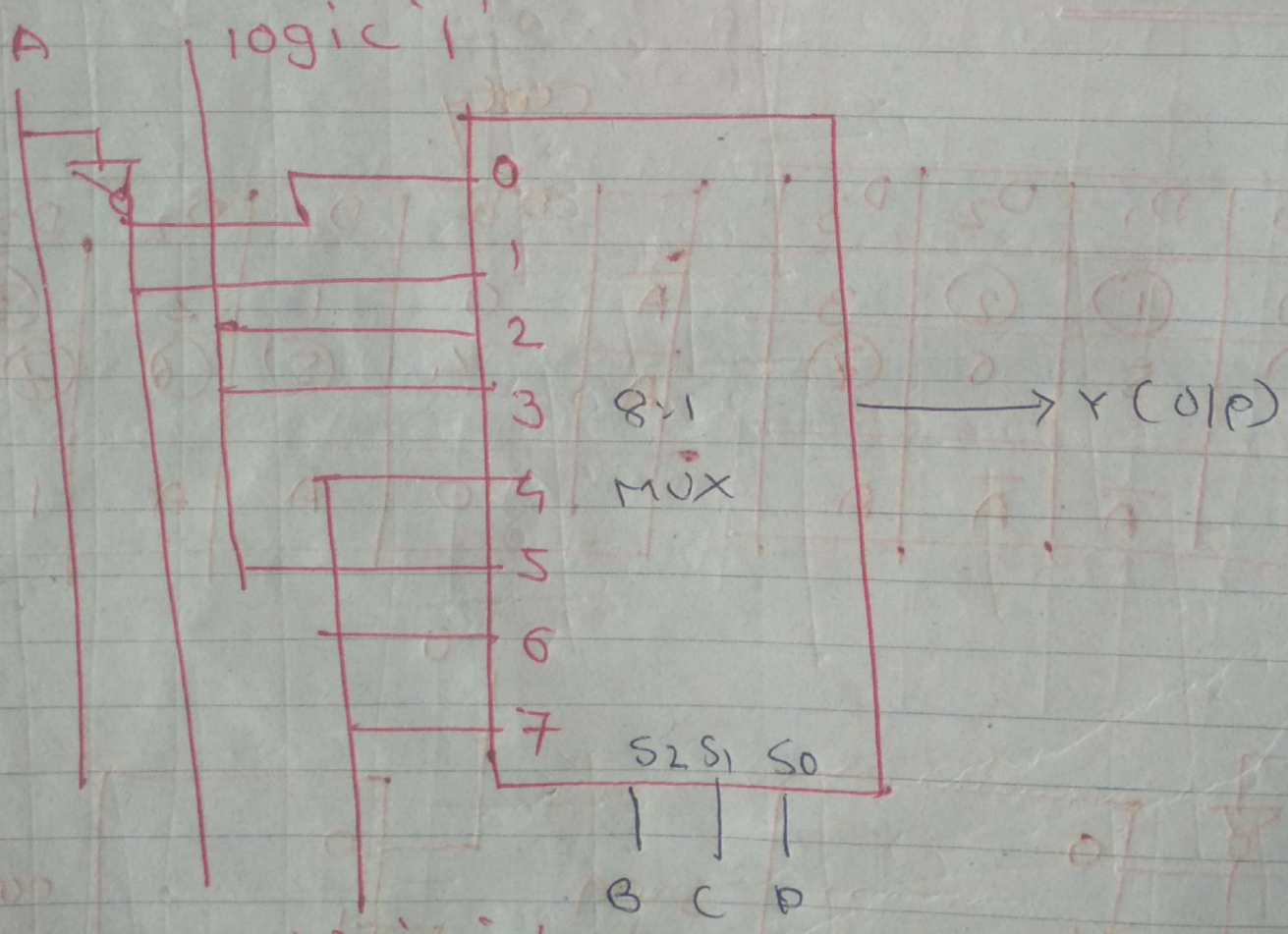
Scanned by CamScanner

(13) Implement the following function using an 8:1 mux.

$f(A,B,C,D) = \Sigma m(1,3,5,10,11,13,14) + d(0,2)$

$\Rightarrow$

	D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7
$\bar{A}$	(0)	(1)	(2)	(3)	4	(5)	6	7
A	8	9	(10)	(11)	12	(13)	14	15
	$\bar{A}$	$\bar{A}$	1	1	0	1	0	0



Implementation:-

https://t.me/SPPU_TE_BE_COMP

Scanned by CamScanner

(D, 24)

Demultiplexer:-

One i/p to many o/p.

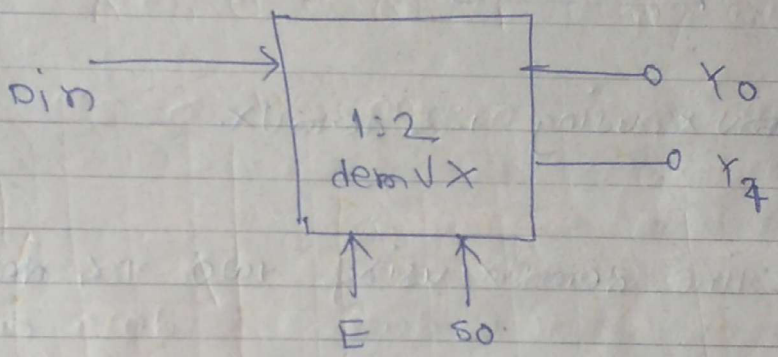
$n = 2^m$ $\quad 8 = 2^m$

no. of o/p $\quad \boxed{m = 3}$

Types of demultiplexer:-

1) 1 : 2 demultiplexer
2) 1 : 4 demultiplexer
3) 1 : 8 demultiplexer
4) 1 : 16 demultiplexer.

I) 1 : 2 demultiplexer.



II) 1 : 4 demux.

4 - o/p
1 - Din
1 - Enable
2 - select lines.

III) 1 : 8 demux.

8 = o/p
1 = Din
1 = Enable
3 = select lines.

IV) 1 : 16 demux

1 = Din
16 = o/p
1 = Enable
4 = select lines.

https://t.me/SPPU_TE_BE_COMP

Scanned by CamScanner

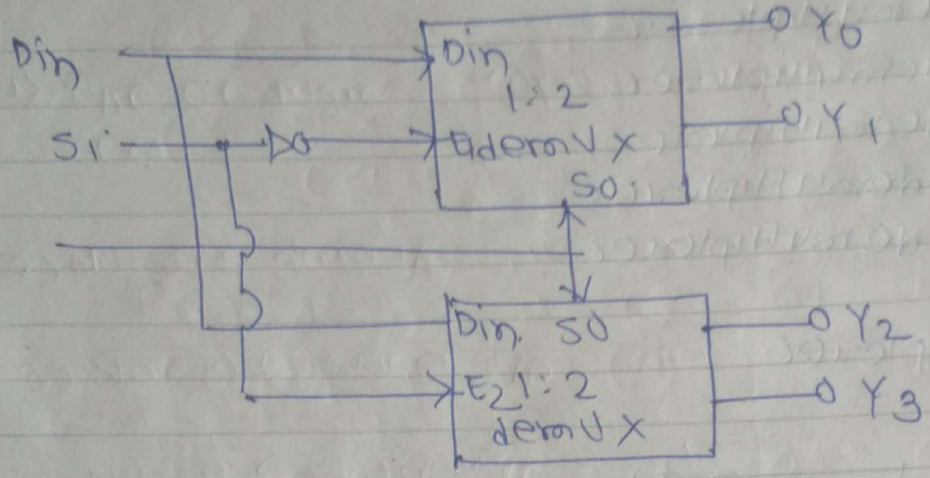
Demultiplexer ICs :-

1) 74138 2) 74139
3) 74154 4) 74155

e.g

I) Obtain 1:4 line demux using 1:2 demux.

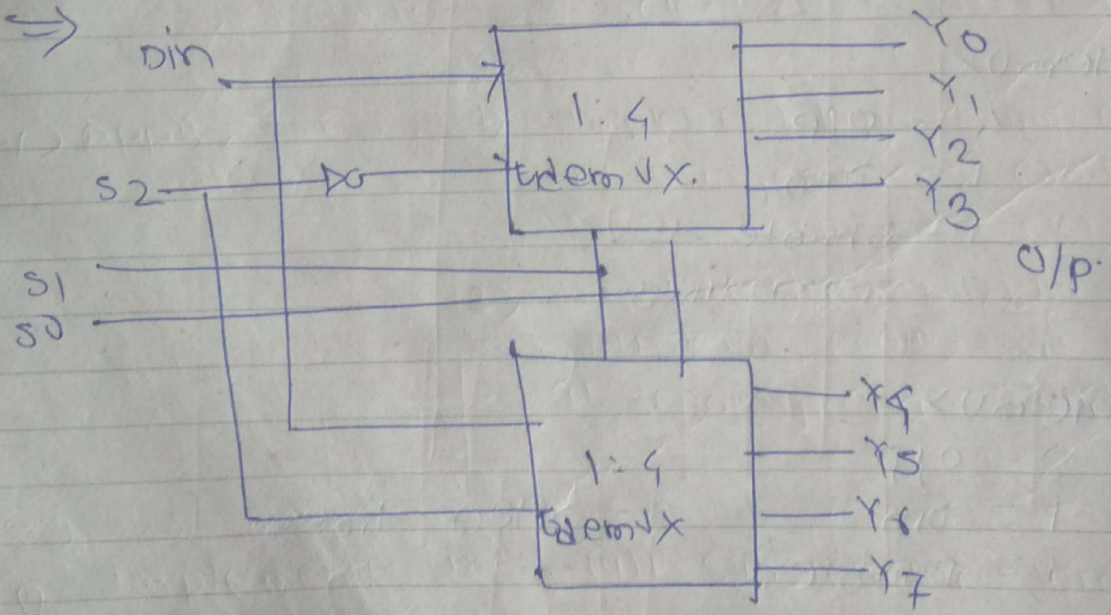
⇒



1:4 demux using 1:2 demux.

II). Obtain 1:8 line demux using two 1:4 demux.

⇒

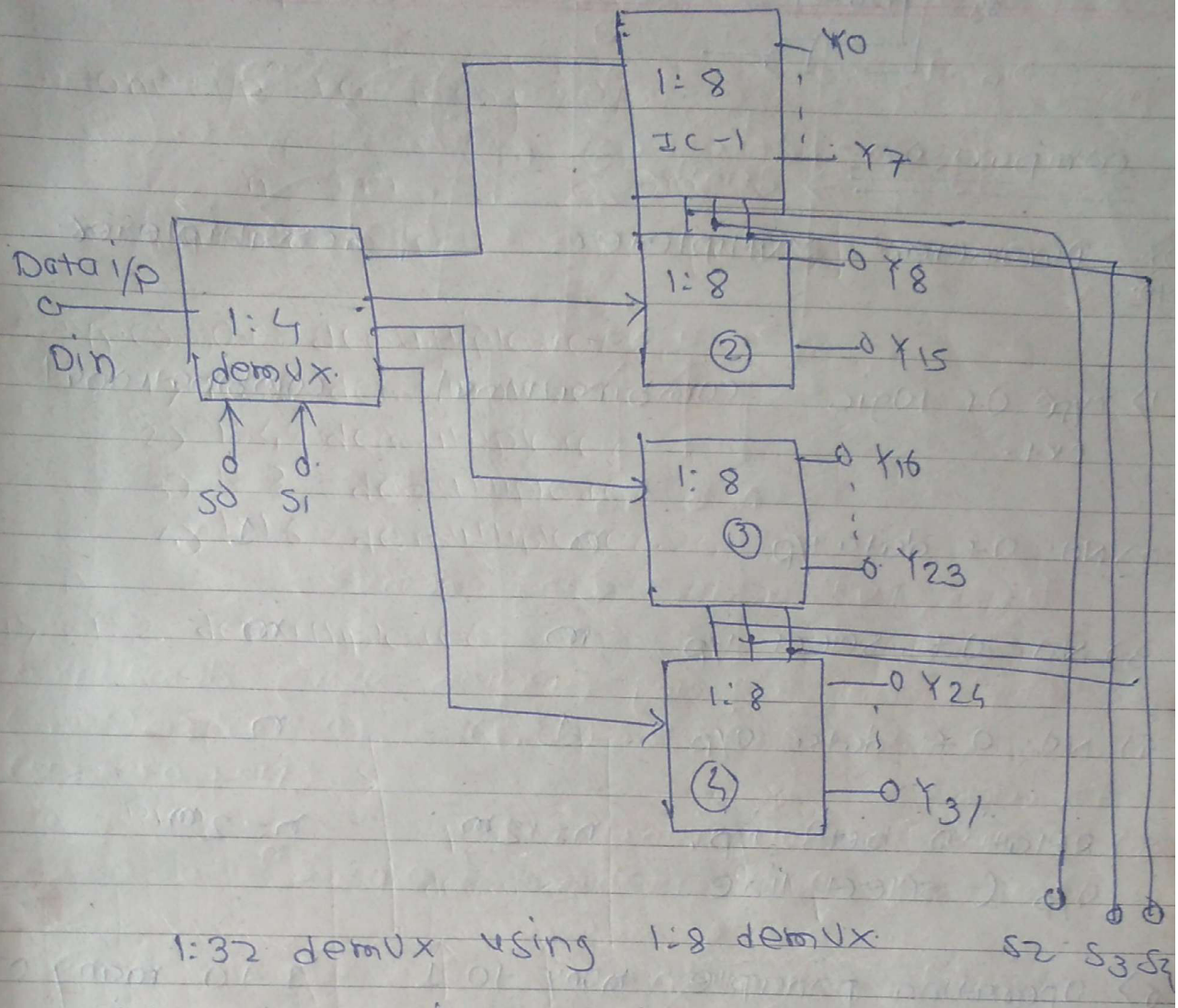


https://t.me/SPPU_TE_BE_COMP

Scanned by CamScanner

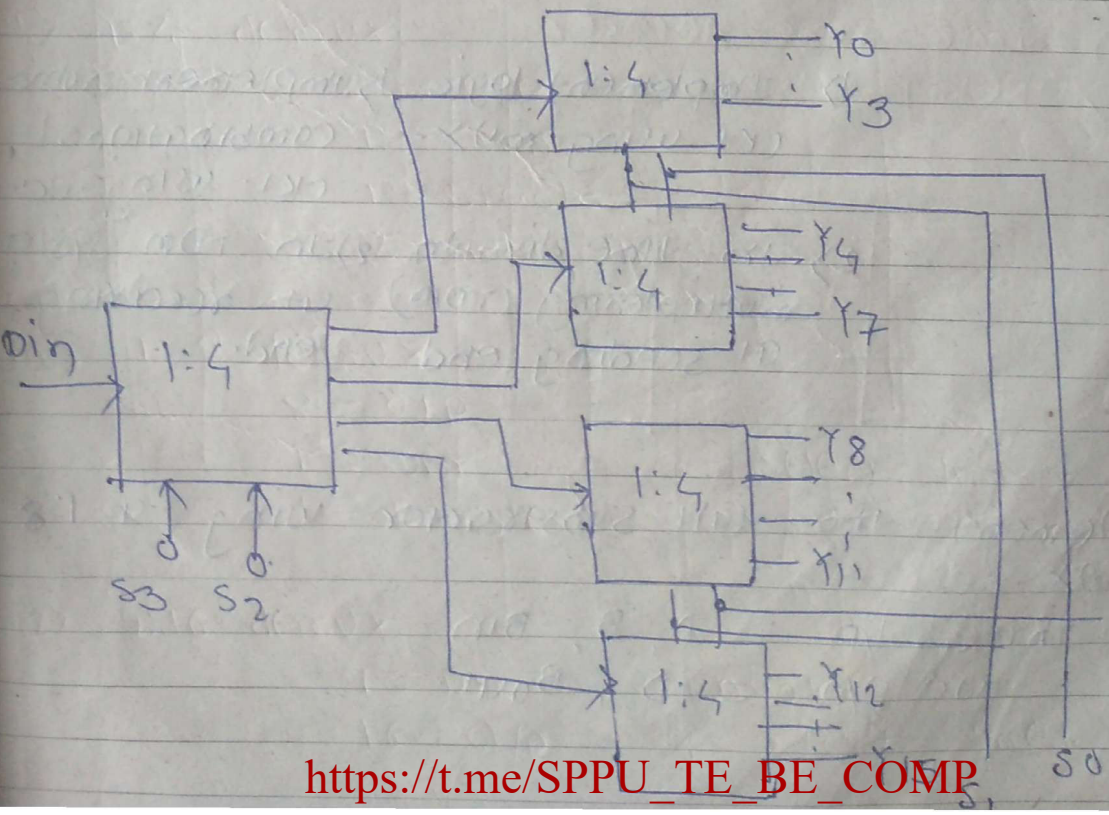
③ Obtain 1:32 demux using 1:8 demux.

⇒



1:32 demux using 1:8 demux

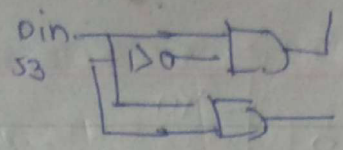
④ 1:16 demux using 1:4 demux.



https://t.me/SPPU_TE_BE_COMP

Scanned by CamScanner

* 1:16 using 1:8 demux.



comparison :-

Sr. No.	Parameter	Multiplexer	demultiplexer
1)	Type of logic ckt.	combinational	combinational
2)	No. of data i/p	n	1
3)	No. of select i/p	m	m
4)	No. of data o/p	1	n
5)	Relation betn i/p, o/p & select line	$n = 2^m$	$n = 2^m$
6)	Operating principle	many to 1 or as data selector.	1 to many or data distributor
7)	Appn.	1) Implement logic ckt. using mux. 2) In time division multiplexing (TDM) at sending end	1) Implement same combinational ckt. using de 2) In TDM syste at receiving end.

e.g

① Implement the full substractor using a 1:8 demux :-

⇒ Three i/p = A, B, Bin
Two o/p = D, Bout.

https://t.me/SPPU_TE_BE_COMP

Scanned by CamScanner

I/p			O/p	
A	B	Bin	D	BOut
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

Express difference & borrow in standard SOP form.

$D = f(A, B, Bin) = \Sigma m(1, 2, 4, 7)$

$B_{OUT} = f(A, B, Bin) = \Sigma m(1, 2, 3, 7)$

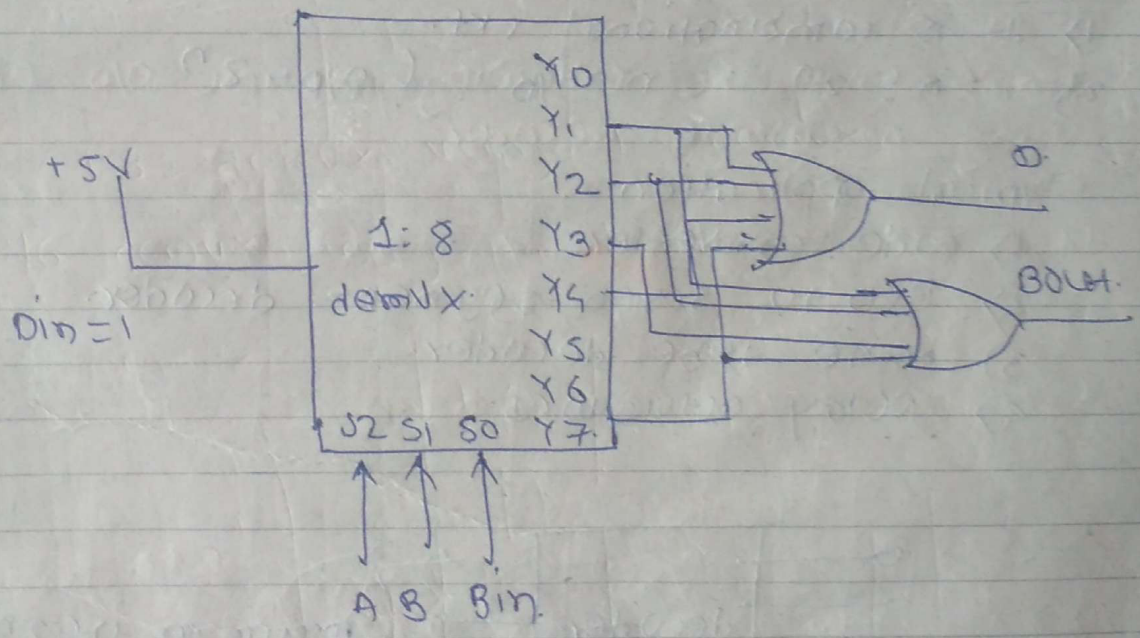


fig: full substractor using 1:8 demux.

(7) Full adder using 1:8 demux.

→ $Sum = \Sigma m(1, 2, 4, 7)$

Carry. = Σm

https://t.me/SPPU_TE_BE_COMP

Scanned by CamScanner

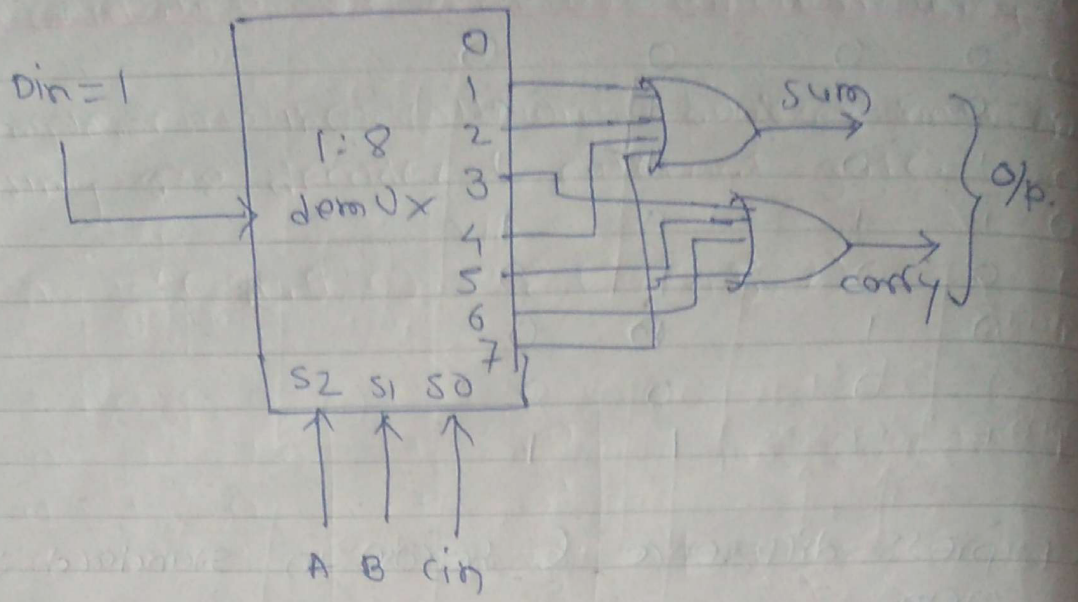


fig :- Full adder using demux.

③ Implement the following function using demultiplexer

$f_1(A,B,C) = \Sigma m(0,3,7)$ $f_2(A,B,C) = \Sigma m(1,2,5)$

Decoder :-

https://t.me/SPPU_TE_BE_COMP

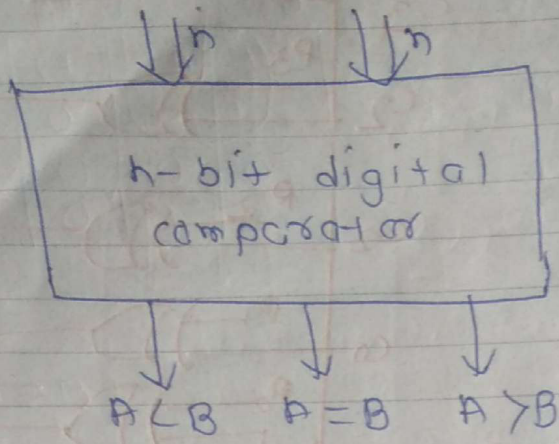
* Magnitude comparator using using 7485

1) Digital comparator is a combinational circuit designed to compare two n-bit binary word.
2) It has 3 o/p namely
   1) A > B
   2) A = B
   3) A < B

https://t.me/SPPU_TE_BE_COMP

Scanned by CamScanner

Block diagram of an n-bit comparator



1-Bit magnitude comparator -

Two i/p = A, B

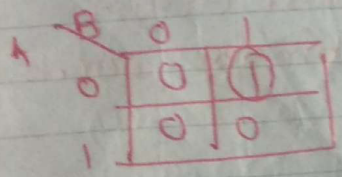
Three o/p = A<B, A=B, A>B

Truth table : one bit comparator.

I/P		O/P		
A	B	$Y_1 = A<B$	$Y_2 = A=B$	$Y_3 = A>B$
0	0	0	1	0
0	1	1	0	0
1	0	0	0	1
1	1	0	1	0

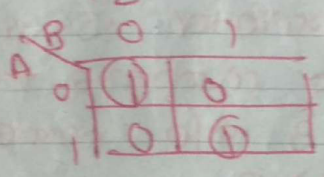
K-map

For Y_1 (A<B)



$Y_1 (A<B) = \bar{A}B$

For Y_2 (A=B)



$Y_2 = A\bar{B} + AB = \overline{A \oplus B}$

https://t.me/SPPU_TE_BE_COMP

Scanned by CamScanner

$Y_3 = A > B$

A \ B	0	1
0	0	0
1	1	0

$Y_3 = A\bar{B}$

Logic diagram

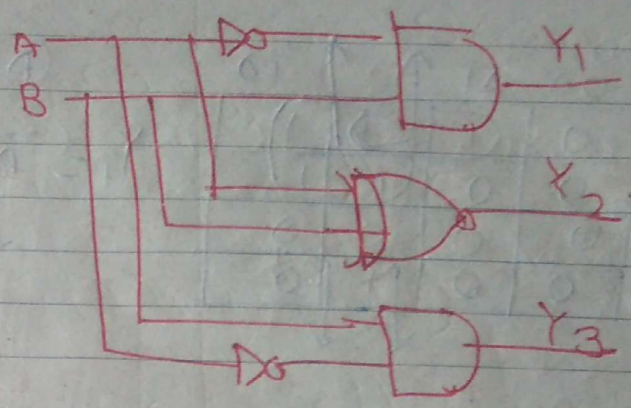


Fig:- Using AND & Ex-OR gate

* 2-bit comparator :-

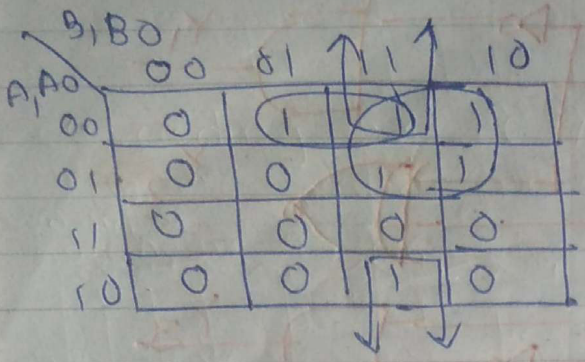
2-bit comparator, each i/p word is 2 bit long.

I/p				O/p		
A_1	A_0	B_1	B_0	$A<B$	$A=B$	$A>B$
0	0	0	0	0	1	0
0	0	0	1	1	0	0
0	0	1	0	1	0	0
0	0	1	1	1	0	0
0	1	0	0	0	0	1
0	1	0	1	0	1	0
0	1	1	0	1	0	0
0	1	1	1	1	0	0
1	0	0	0	0	0	1
1	0	0	1	0	0	1
1	0	1	0	0	1	0
1	0	1	1	1	0	0
1	1	0	0	0	0	1
1	1	0	1	0	0	1
1	1	1	0	0	0	1
1	1	1	1	0	1	0

https://t.me/SPPU_TE_BE_COMP

Scanned by CamScanner

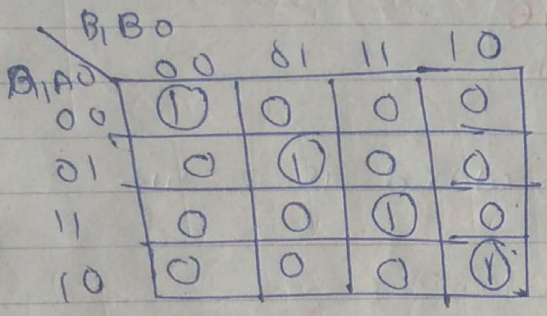
$Y_1 (A<B)$



$A<B$

$Y_1 = \overline{A_1} \cdot B_1 + \overline{A_1}\,\overline{A_0} \cdot B_0 + \overline{A_0} \cdot B_1 B_0$

$Y_2 (A=B)$



$Y_2 (A=B)$

$= \overline{A_1}\,\overline{A_0} \cdot \overline{B_1}\,\overline{B_0} + \overline{A_1} A_0 \cdot \overline{B_1} B_0 + A_1 A_0 B_1 B_0 + A_1 \overline{A_0} B_1 \overline{B_0}$

$= \overline{A_0}\,\overline{B_0}(\overline{A_1}\,\overline{B_1} + A_1 B_1) + A_0 B_0 (\overline{A_1}\,\overline{B_1} + A_1 B_1)$

$= \overline{A_0}\,\overline{B_0} (A_1 \overline{\oplus} B_1) + A_0 B_0 (\overline{A_1 \oplus B_1})$

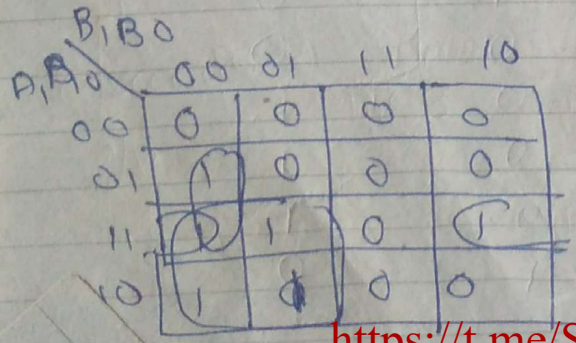
$= (A_1 \overline{\oplus} B_1) \cdot (\overline{A_0}\,\overline{B_0} + A_0 B_0)$

$= (\overline{A_1 \oplus B_1})\,(\overline{A_0 \oplus B_0})$

$$\boxed{Y_2 (A=B) = (A_1 \odot B_1)(A_0 \odot B_0)}$$

$\odot$ = Ex-NOR

$Y_3 (A>B)$



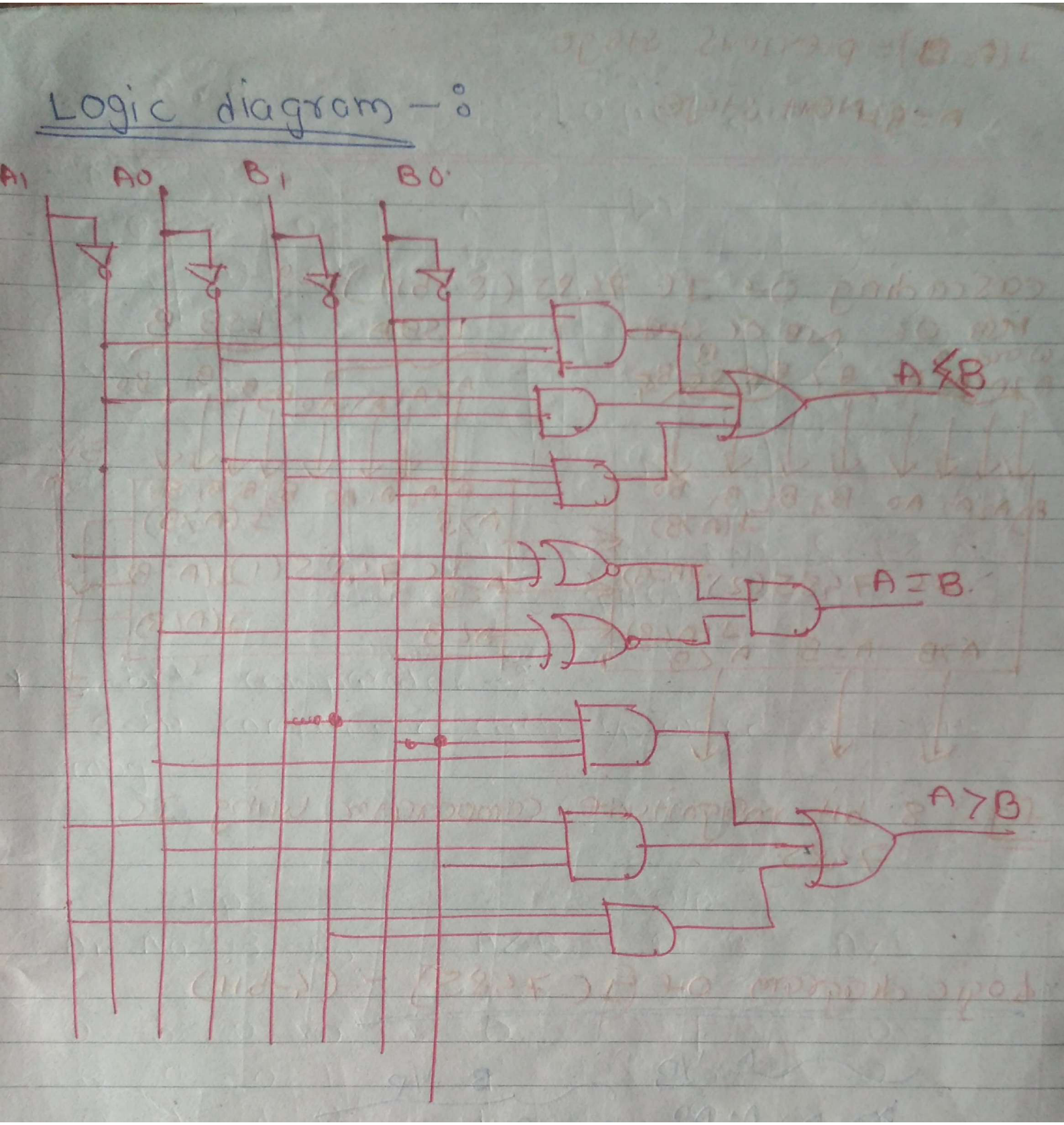
$Y_3 (A>B)$

$= A_1 \cdot \overline{B_1} + \overline{B_1}\,\overline{B_0} \cdot A_0 + A_1 A_0 \cdot \overline{B_0}$

https://t.me/SPPU_TE_BE_COMP

Scanned by CamScanner

Logic diagram :-



https://t.me/SPPU_TE_BE_COMP

Scanned by CamScanner